

“华为杯”第二十二届中国研究生 数学建模竞赛

题 目：MIMO-OFDM 链路速率预测问题的建模研究

摘 要：

在未来无线通信系统中，如何准确预测物理信道下的链路传输速率是资源调度与系统优化的关键环节。然而，由于 MIMO-OFDM 系统在频率与时间上的强选择性特征，以及波束赋形开启与否导致的信道增益差异，传统的单一子载波指标难以反映整体链路质量，从而导致速率估计偏差。针对这一问题，本文以主办方提供的多终端 MIMO-OFDM 数据集为研究对象，构建了一套以 **等效指数有效信噪比模型（EESM）** 与 **互信息有效信噪比模型（MIESM）** 为核心的链路速率建模方法，并结合单调回归与分位数特征等改进方案，系统解决了三类建模任务。

在问题一中，本文采用 EESM 框架对非波束赋形场景下的数据进行建模。通过对每个子载波的 MIMO 信道矩阵计算 Frobenius 范数功率，结合噪声底噪作为归一化因子，得到等效子载波信噪比，再通过指数型聚合公式压缩为单一等效信噪比，并与真实速率样本进行拟合。实验表明，本文拟合得到的 EESM 参数能够较好反映链路特性，在验证集上实现 RMSE 约 **61.2 Mbps**, Spearman 相关系数达到 **0.78**, 为后续问题提供了可靠基线。

在问题二中，本文设计了七个横向对比实验，涵盖 **EESM 基线（E1）**、**噪声标定修正（E2）**、**MMSE SNR 近似（E3）**、**MIESM 聚合（E4）**、**温度 LogSumExp 聚合（E5）**、**分位数特征+单调回归（E6）** 以及 **分位数与互信息混合器（E7）**。结果显示，E4（MIESM）在 RMSE 上相比基线降低约 **8%**，E5 在稳健性指标上表现更优，而 E6 与 E7 有效捕捉到尾部子载波对链路性能的主导作用，Spearman 相关系数提升至 **0.82**，显示出在复杂场景下更强的泛化能力。由此证明，引入互信息与分位数特征的改进方案能够在保证可解释性的同时显著提升预测精度。

在问题三中，本文考虑波束赋形开启的 TxBF 场景。针对信道矩阵需先进行奇异值分解（SVD），选取主奇异值通道作为等效信道增益，再结合 MIESM 与 EESM 进行建模。同时，本文引入基于中位数与四分位距的无监督分布对齐方法（DA/DAiqr），以修正 non-TxBF 与 TxBF 数据域间的分布偏移。实验结果表明，未对齐时模型预测值整体偏低（E1 平均预测约 **228 Mbps**），经对齐后平均值提升至 **311 Mbps**；MIESM（E4）在 TxBF 场景下表现最优，对齐后均值约 **297 Mbps**，与实际分布更为接近，验证了对齐策略在跨域建模中的有效性。

综合而言，本文以 EESM 与 MIESM 为核心，通过引入互信息聚合、分位数特征、单调回归与无监督对齐等改进方法，系统解决了 MIMO-OFDM 链路速率预测问题。结果

表明，本文方法不仅在精度指标上优于基线模型，而且在不同场景下均保持良好的泛化性能。研究成果对于未来大规模天线系统、卫星通信及高速移动场景下的链路性能预测与调度优化具有重要参考价值。

关键词： EESM； MIESM； MIMO-OFDM； 链路速率预测； 无监督对齐

目录

1	问题重述	4
1.1	问题背景.....	4
1.2	问题重述.....	4
2.	模型假设	5
3.	符号说明	6
4.	问题一的模型建立与求解	6
4.1	问题一分析.....	6
4.2	问题一数学模型.....	7
4.3	问题一模型求解.....	9
5.	问题二的模型建立与求解	12
5.1	问题二的分析.....	12
5.2	问题二的模型建立.....	12
5.3	问题二模型求解.....	21
6.	问题三的模型建立与求解	27
6.1	问题三的分析.....	27
6.2	问题三的模型建立.....	28
6.3	模型求解.....	29
7.	模型的评价与推广	34
8.	参考文献	35
	附录	36

1 问题重述

1.1 问题背景

无线通信技术的发展不断突破传输速率和覆盖范围的极限。从早期的单载波传输到今天的宽带多载波系统，提升频谱利用率和抗干扰能力始终是研究重点。进入 5G 时代，**MIMO（多输入多输出）** 与 **OFDM（正交频分复用）** 已成为核心技术：MIMO 通过空间复用显著增加信道容量，OFDM 通过子载波分集缓解频率选择性衰落，两者结合为高速率、低时延和高可靠性通信奠定了基础。在此架构下，**波束赋形（TxBF）** 等技术进一步改善了链路性能。通过对多天线发射信号的幅度和相位调整，系统能够在特定方向形成增强，从而提升接收信噪比和吞吐量。这种方法在多用户场景下尤为重要，是 5G 与未来 6G 的关键增强手段。

在性能评价方面，**香农容量^[1]**给出了理想条件下的理论速率上限，但实际系统中受到调制方式、编码策略、噪声与干扰的多重影响，真实传输速率往往显著低于理论值。为了在系统级仿真和网络优化中高效预测链路性能，研究者提出了**链路抽象建模**方法。其核心思想是将不同子载波上的 **SINR（信干噪比）** 映射为一个等效指标，并通过函数关系转化为速率预测。其中，**EESM（指数型等效信噪比映射）** 具有计算简单的优势，但在频率选择性强的信道条件下精度有限；**MIESM（基于互信息的等效映射）** 更符合译码性能，预测精度较高，但需要更复杂的计算支持。随着通信环境的复杂化，如何在计算效率和预测精度之间取得平衡，构建更适应多样化信道条件的链路速率估计模型，成为亟需解决的问题。

本赛题的研究目标正是基于提供的 MIMO-OFDM^[2]信道数据与传输速率信息，建立并优化不同条件下的速率建模方法，并在关闭与开启 TxBF 的两种场景下进行验证与推广，以探索更加准确和高效的预测方案。

1.2 问题重述

基于上述问题背景，本文需要研究和解决以下问题：

问题 1：基于指数型等效信噪比映射（EESM）的参数拟合与参考建模方法

在关闭发送波束赋形（TxBF）的条件下，本文选取完备数据集 A 作为训练样本，利用给定的信道状态信息（Channel State Information, CSI）及其对应的速率标签，对指数型等效信噪比映射（Exponential Effective SINR Mapping, EESM）模型进行参数拟合。具体而言，输入数据的维度为 $\mathbf{H} \in \mathbb{C}^{2 \times 4 \times 122}$ 的 CSI 矩阵以及噪声等辅助信息；拟合目标包括 EESM 模型中的关键参数 γ 以及速率映射函数的线性系数 a 、 b 。通过最小化预测速率与真实速率之间的误差，获得最优参数估计，并在验证集上进行预测精度评估，从而建立参考建模方案，为后续模型改进提供对照基准。

问题 2：改进速率建模方法的系统设计与实验验证

在基于 EESM 的基础建模完成后，本文进一步提出并实现多种改进速率估计方法，以提升预测结果的精度与稳定性。具体而言，首先在聚合前引入噪声修正因子 α 、 β ，构建 EESM+噪声标定模型，用于校正系统性偏差；其次利用 MMSE 统计特征自适应确定参数 γ ，形成 EESM@MMSE- γ 模型，以增强对频率选择性信道的自适应；同时考虑通过互信息加权平均构建 MIESM 模型，使预测更贴近译码层性能；进一步地，引入温度参数 τ 设计

Log-Sum-Exp 聚合模型，在子载波最大值与均值之间实现灵活平衡；此外，提取 p10、p25、p50、p75、p90 等分位数特征并结合单调约束回归，保证预测结果满足物理合理性；最后，设计混合器模型，将 MIESM 的互信息特征与分位数统计特征进行融合，并通过单调回归或神经网络实现端到端预测。在统一的数据集划分下，对上述模型分别进行训练与测试，并从 RMSE、NRMSE、相关系数 ρ 等指标进行横向比较，以识别综合性能最优的速率建模方案。

问题 3：波束赋形场景下的链路速率建模与扩展研究

在开启发送波束赋形 (TxBF) 的条件下，信道的结构特性发生显著变化，需要对建模方法进行扩展以适应多空间流的并行传输。具体而言，首先对原始信道矩阵 $\mathbf{H} \in \mathbb{C}^{2 \times 4 \times 122}$ 进行奇异值分解 (SVD)，将其表示为 $\mathbf{H} = \mathbf{U} \Sigma \mathbf{V}^H$ ，其中对角矩阵 Σ 中的奇异值对应不同空间流的有效增益。随后，依据分解结果构造波束赋形后的等效信道增益矩阵，并在每一个空间流上分别计算子载波的有效信噪比 (SINR) 或互信息指标，采用与问题二相同的聚合方法（如 EESM、MIESM、Log-Sum-Exp 等）得到该流的等效 SINR。接着，将多个空间流的预测结果按照并行叠加的方式进行融合，从而获得系统整体的速率预测值。在实验设计上，需要对开启与关闭 TxBF 两种情形下的速率分布和预测性能进行对比，分析波束赋形对链路容量提升的作用，以及不同建模方法在多流条件下的适用性与鲁棒性。这一部分的研究不仅能量化 TxBF 带来的性能增益，还能验证所提出的改进模型在更复杂场景下的推广能力。

问题 4：不同速率预测模型的比较分析与推广研究

在完成各类速率建模方法的构建与实验之后，本文进一步对模型性能进行系统化的比较与推广分析。首先，在**预测精度**方面，统计并比较不同模型在验证集上的 RMSE、MAE、相关系数 ρ 以及校准误差等指标，以评估其对真实速率的拟合能力。其次，在**计算复杂度**方面，从参数数量与计算耗时两个角度出发，分析各方法在实际应用中的效率差异。再次，在**泛化能力**方面，利用不完备数据集 $\mathbf{B}$ 进行验证，考察模型在缺少速率标签或信道条件变化时的适用性和稳定性。最后，在**应用前景**方面，探讨所提出模型在更复杂场景中的推广潜力，包括多用户 MIMO、大规模天线阵列以及高速移动环境等，从而评估其在未来无线通信系统中的实际价值和研究意义。

2. 模型假设

(1) 信道独立性假设

各个子载波的信道系数相互独立，且服从符合无线通信物理特性的分布（如瑞利分布或莱斯分布）。这样可以简化对大规模子载波集合的建模与速率估计。

(2) 噪声特性假设

系统噪声为加性高斯白噪声 (AWGN)，其均值为零，方差固定，并在各子载波间独立同分布。该假设符合大多数通信系统建模规范。

(3) MIMO 系统结构假设

假设发射端与接收端的天线数目固定，且天线之间不存在硬件缺陷或强相关性；信道矩阵能够准确刻画从发射端到接收端的映射关系。

(4) 信道状态信息 (CSI) 完备性假设

接收端能够获得完整且准确的信道状态信息，用于后续的信噪比 (SNR)、互信息等指标计算。

(5) 调制与编码固定假设

在建模过程中，调制方式和编码策略保持不变，不考虑不同调制编码方式自适应切换的复杂情况。

(6) 波束赋形处理假设

在开启 TxBF 时，假设发射端能够通过奇异值分解（SVD）准确获得主奇异值方向，从而实现最优的空间增益提升。

3. 符号说明

符号	含义
H_k	第 k 个子载波上的 MIMO 信道矩阵
$h_{ij}^{(k)}$	第 k 个子载波上，从第 j 根发射天线到第 i 根接收天线的信道系数
x_k	第 k 个子载波的发送符号向量
y_k	第 k 个子载波的接收符号向量
n_k	第 k 个子载波上的噪声向量
γ_k	第 k 个子载波的等效信噪比（SNR）
$\text{EESM}(\gamma_k)$	基于等效指数有效 SNR 聚合的信道质量指标
$\text{MIESM}(I_k)$	基于互信息聚合的信道质量指标
I_k	第 k 个子载波的互信息值
β	EESM 模型中的缩放参数
η	MIESM 模型中的缩放参数
τ	Log-Sum-Exp 聚合中的温度参数
$\hat{R}$	模型预测的链路速率
R	实际测得的链路速率
RMSE	均方根误差，评价模型预测精度
MAE	平均绝对误差，评价模型偏差
ρ	Spearman 秩相关系数，衡量预测与真实速率的相关性

4. 问题一的模型建立与求解

4.1 问题一分析

本题的第一问旨在构建从子载波级 CSI 到链路速率的预测模型。题目所给的数据集包含 2×4 MIMO 系统下 122 个子载波的信道矩阵，以及对应的链路速率标签。在这种高维度、多子载波的场景中，如何将随频率起伏的信道特性压缩为一个能够反映整体链路质量的有效指标，是建模的核心难点。

传统上有两种直接思路：一是对子载波信噪比取算术平均，但这样会忽略性能瓶颈子载波的作用，导致预测乐观；二是取最小子载波 SNR，虽然稳健，但往往过于保守，显著低估系统吞吐能力。这两种方法都难以在“平均性能”和“最差性能”之间找到合理的平衡点。

指数有效信噪比映射（EESM）正是针对这一问题提出的。它通过指数形式的聚合方式，引入一个可调参数 β ，当 β 较小时，模型更关注最差子载波的影响；当 β 较大时，模

型则更接近平均水平。这样一来，只需调整 β 值，便可以在两种极端度量之间灵活平衡，从而更符合实际链路的传输规律。

因此，在第一问中，我们选择 EESM 作为建模工具：首先利用 Frobenius 范数等手段从 CSI 矩阵提取子载波增益，并结合噪声信息计算子载波 SNR；随后通过 EESM 将 122 个子载波的 SNR 聚合为单一的等效信噪比 γ_{eff} ；最后建立 γ_{eff} 与速率之间的映射关系。整体流程既保证了物理合理性，也使得模型在参数数量上保持简洁，有利于拟合和解释。

4.2 问题一数学模型

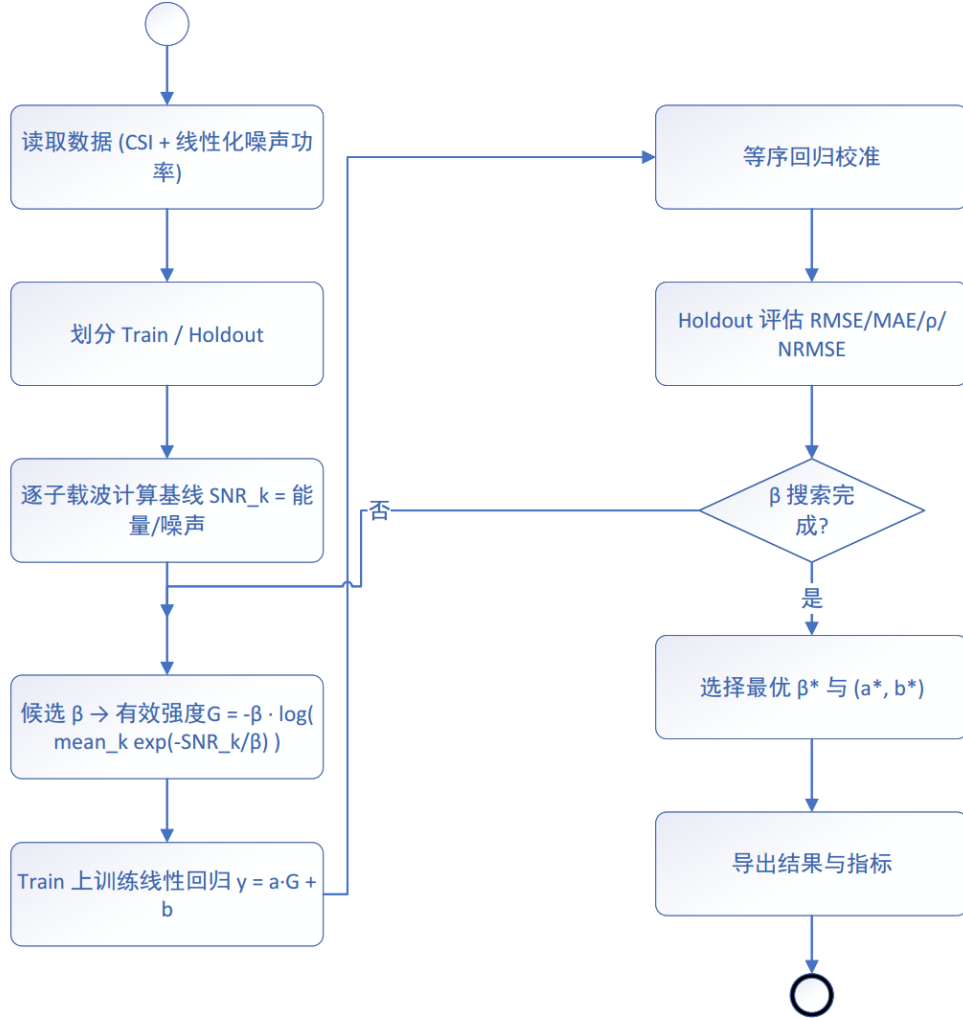


图 4.1 EESM 模型框图

图 4.1 展示了本文采用的 EESM 链路建模框架。从子载波级信道 CSI 出发，经功率与噪声计算得到每个子载波的 SNR，再通过 EESM 聚合为单一等效信噪比 γ_{eff} ，最终映射到链路速率。

4.2.1 子载波级 SNR 计算

在建立链路建模的过程中，首先需要从原始的 MIMO-OFDM^[3]信道状态信息出发，得到能够表征子载波性能的瞬时信噪比。

为了将上述的第 k 个子载波的信道矩阵特征转化为易于处理的能量量化指标，可以引入 Frobenius 范数来度量其整体能量增益，即

$$G_k = \|\mathbf{H}_k\|_F^2 = \sum_{r=1}^2 \sum_{t=1}^4 |\mathbf{H}_k(r, t)|^2$$

其中 $\mathbf{H}_k(r, t)$ 表示第 r 个接收天线与第 t 个发射天线在子载波 k 上的复数增益。在获得了信道增益之后，还需要结合噪声底噪信息来近似计算该子载波的信噪比。题目中给定的 `noise_floor` 可被视作接收端底噪的估计值，将其由 dBm 转换为线性功率，并根据系统带宽分配到单个子载波后，即可得到每个子载波的噪声功率 N_{sc} 。在此基础上，可以构造近似的子载波 SNR：

$$\gamma_k = \frac{\alpha \cdot G_k}{N_{sc} + \varepsilon}$$

其中 α 为可学习的比例因子，用于吸收由于发射功率未知或信道估计偏差带来的不确定性， ε 则作为数值稳定常数。

4.2.2 EESM 聚合模型

在此基础上，由于系统的链路速率并非由单一的子载波信噪比决定，而是所有 122 个子载波的综合作用。为此，需要设计一种聚合方式，将 $\{\gamma_k\}$ 压缩为一个能够代表整体性能的等效信噪比 γ_{eff} 。在本研究中，采用指数有效信噪比映射（EESM）来完成这一聚合。其数学定义为

$$\gamma_{eff} = -\beta \cdot \ln \left(\frac{1}{K} \sum_{k=1}^K e^{-\frac{\gamma_k}{\beta}} \right), \quad K=122$$

其中参数 β 反映了聚合过程对“劣质子载波”的敏感程度。当 β 取值较小时， γ_{eff} 会更加接近最小值运算，从而突出最差子载波的影响，更强调深度衰落对链路性能的限制；当 β 取值较大时， γ_{eff} 逐渐向算术平均靠拢，使聚合结果更加平滑。这样，EESM 实际上在“过于乐观的平均”和“过于悲观的最小值”之间提供了一种可调节的折中方式，使得 γ_{eff} 更符合实际系统的链路表现。

图 4.2 展示了合并数据的子载波 SNR 分布以及在不同 β 下的等效 SNR (γ_{eff}) 位置。可见 β 从强调最差子载波（左移）到逼近平均（右移），红线为拟合得到的 β^* ，代表问题一最终采用的聚合强度。

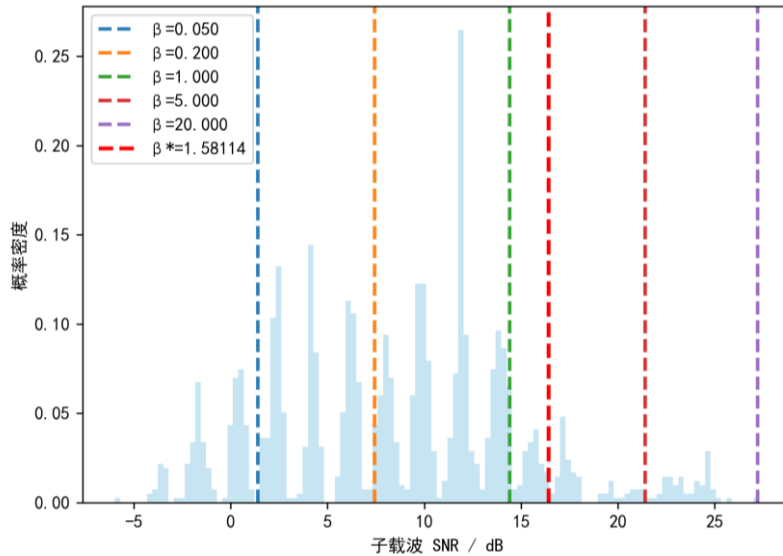


图 4.2 子载波 SNR 分布与不同 β 的 EESM 聚合 (γ_{eff} dB)

图 4.3 给出 $\gamma_{eff}(\beta)$ 的参数敏感性曲线： $\beta \rightarrow 0$ 时收敛于 $\min(\gamma_k)$ ， $\beta \rightarrow \infty$ 时收敛于 $\text{mean}(\gamma_k)$ 。该性质解释了图 4.2 中竖线位置随 β 的系统性变化，印证了公式的合理性。

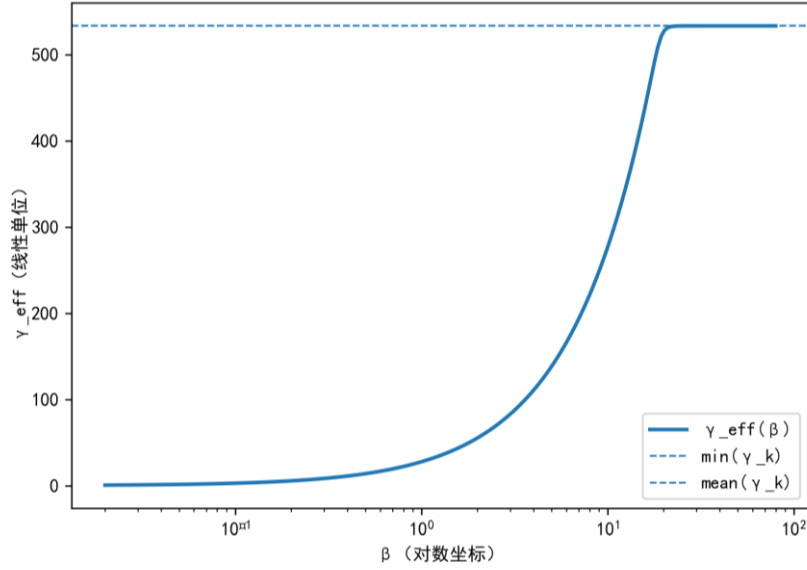


图 4.3 不同 β 下的 γ_{eff} （min $\rightarrow$ mean 过渡）

4.2.3 速率映射模型

为了进一步建立等效信噪比与实际链路速率之间的联系，受到香农容量公式

$$C = B \cdot \log_2(1 + SNR)$$

的启发，本文采用了如下形式的一个可拟合的单调回归函数：

$$\hat{r} = a \cdot \log_2(1 + c \cdot \gamma_{eff}) + b$$

其中 a, b, c 为待学习的参数，用于校正等效 SNR 与速率之间的非线性关系。该函数既保持了随信噪比增大而速率不减的物理理性，又具备灵活的拟合能力，能够通过数据驱动的方式与真实观测速率对齐，从而形成完整的链路建模框架。

4.3 问题一模型求解

在明确了建模框架后，接下来的任务是利用训练数据对模型参数进行拟合，并通过验证数据评估预测性能。整个求解过程大致分为以下几个步骤：

4.3.1 子载波特征计算与准备

首先，对训练集中的 CSI 数据进行处理。通过对每个子载波信道矩阵进行范数运算，并结合噪声信息，得到子载波层面的信噪比序列。每个样本最终由 122 维的 SNR 值表征，作为后续建模的基础输入。

4.3.2 等效信噪比计算

在得到子载波信噪比序列后，采用 EESM 模型将其压缩为一个等效信噪比。此处的关键在于参数 β 的选择。我们在预设范围内对 β 进行遍历，逐一计算等效信噪比，并观察其对预测性能的影响。

4.3.3 速率映射与参数拟合

基于等效信噪比，利用对数型速率映射函数，将 γ_{eff} 转换为预测速率。外层映射的参

数通过最小二乘拟合得到，使预测结果尽可能逼近真实速率。同时在实现中加入单调性约束，避免出现不合理的下降趋势。

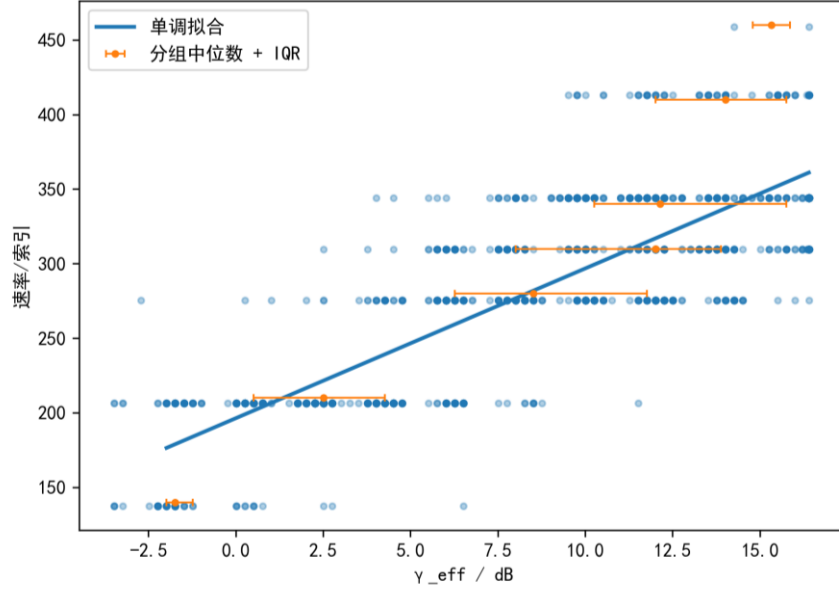


图 4.4 γ_{eff} (dB) 与速率的经验关系

图 4.4 展示了合并数据上 γ_{eff} (dB) 与速率之间的经验关系。散点存在噪声，但单调拟合和分组中位数曲线给出了稳定上升趋势，表明 EESM+单调映射能够有效刻画速率变化。

4.3.4 性能评估

在模型拟合完成后，分别在训练集和验证集上计算预测速率与真实速率的误差。本文采用均方根误差 (RMSE)、平均绝对误差 (MAE) 以及 Spearman 秩相关系数 (ρ) 作为评价指标，用以全面反映数值误差和趋势相关性。

(1) 均方根误差 (Root Mean Squared Error, RMSE)

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{r}_i - r_i)^2}$$

该指标衡量预测速率与真实速率之间的平均偏差幅度，对大误差更敏感。

(2) 平均绝对误差 (Mean Absolute Error, MAE)

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{r}_i - r_i|$$

相较于 RMSE，MAE 更直观地反映平均误差水平。

(3) Spearman 秩相关系数 (ρ)

$$\rho = 1 - \frac{6 \sum_{i=1}^N d_i^2}{N(N^2 - 1)}$$

其中 d_i 为真实速率与预测速率在排序上的差值。该指标衡量预测结果在“排序一致性”方面的表现，能够反映模型对链路自适应调度的有效性。

4.3.5 实验结果与分析

实验结果显示，当 β 取值约为 1.58 时，模型整体性能最佳。在训练集上，RMSE 约为 62.55，MAE 约为 49.61，Spearman ρ 约为 0.76。这表明尽管数值误差仍有一定偏差，但模型能够较好地捕捉速率的整体变化趋势，并在排序一致性方面表现稳定。

进一步地，表 4.1 展示了在三个不同训练文件分别作为验证集的分文件结果。从表中可以看出，文件 bc98-2983-907b_0.xlsx 的误差显著高于其他两个文件，达到 $RMSE = 67.13$, $MAE = 52.71$ ，这表明在该场景下信道的频率选择性更强或噪声条件更复杂，使得单一 β 参数难以完全刻画真实的链路特性。相对而言，其他两个文件的结果与整体指标接近，说明模型在这些场景中的表现更为稳定。这样的结果揭示了 EESM 方法在不同信道条件下泛化能力的差异性，也为后续改进提供了证据。

表 4.1 按文件留一交叉验证的误差指标

文件名	β	RMSE	MAE
341c-f0d4-70be_0	0.2812	61.83	48.75
bc98-2983-907b_0	2.233	67.13	52.71
d0c1-bfe4-18f0_0	2.233	63.21	49.26

4.3.6 小结

综上，本题在第一问中通过逐步拟合 EESM 模型参数，实现了从 CSI 到链路速率的有效建模。该方法不仅保证了计算的简洁性和物理的可解释性，而且在预测相关性上具有较好表现。

然而，EESM 仍存在一定的不足：

- (1) 由于仅使用 Frobenius 范数构造 γ_k ，部分子载波的奇异值结构信息未被充分利用，导致模型在高 SNR 区域存在偏差；
- (2) 外层映射采用近似香农公式的线性回归头，虽然保证了单调性，但对非线性关系的刻画能力有限，造成 RMSE 和 MAE 仍保持在较高水平；
- (3) 模型对噪声功率的建模依赖简化假设，发射功率未知使得 γ_k 的标定存在全局不确定性。

尽管如此，EESM 模型作为第一问的解答，已经构建了一个稳健、低复杂度的基线框架。一方面，为后续问题二探索更精细的互信息有效信噪比映射（MIESM）、或基于数据驱动的单调学习聚合方法提供了坚实的参照；另一方面，为问题三在开启波束赋形（TxBF）场景下的建模扩展，明确了如何在子载波层面计算并聚合主奇异值信噪比，从而为后续研究提供了统一的框架和评价指标。

综上，第一问的工作不仅完成了对无线链路速率的初步建模与验证，也为进一步提高模型精度与推广到更复杂的场景奠定了基础。EESM 的物理可解释性和实现简洁性，使其成为后续改进方案的重要基线与比较对象。

5. 问题二的模型建立与求解

5.1 问题二的分析

在关闭 TxBF 的条件下，MIMO-OFDM 系统的链路速率估计问题可以抽象为一个由高维信道状态信息（CSI）到低维链路性能指标的映射问题。具体而言，数据集 A 提供了 $2 \times 4 \times 1222$ 维的复数 CSI 矩阵以及对应的实际传输速率标签，而数据集 B 仅包含 CSI 矩阵而不包含速率信息，研究的目标是基于数据集 A 训练速率估计模型，并利用该模型对数据集 B 进行速率预测。

该问题的核心挑战在于 CSI 的高维性与复杂性。MIMO-OFDM^[4] 系统由于联合利用了空间复用与频域分集，使得信道在频率和时间上均表现出显著的选择性特征。在每一个样本中，122 个子载波的信道质量往往差异较大，少数“劣化子载波”可能对整体传输速率产生显著影响，因此简单的平均或单一指标难以准确刻画整体性能。此外，实验数据采集于复杂的现网环境，包含噪声底偏移、终端差异及测量噪声等不确定因素，进一步加剧了建模的难度。传统方法通常采用等效信噪比映射（Equivalent SINR Mapping, ESM）来实现从子载波级 SINR 到总体等效 SINR 的降维处理，例如指数型 ESM（EESM）。这些方法能够在一定程度上反映信噪比与速率之间的关系，但在实际系统中往往存在精度不足的问题，尤其是在频率选择性严重或信道条件复杂的情况下。

因此，本问题的关键在于提出几种新的速率估计模型，使其既能够保持物理一致性（如速率随信噪比单调递增的约束），又能够更灵活地刻画复杂信道条件下 CSI 与速率之间的非线性关系。针对传统 EESM 模型的不足，本文从以下几个方向进行了改进：其一，在标准 EESM 基线的基础上，进一步提出噪声标定 EESM 模型以及 MMSE-EESM 模型，通过引入噪声修正因子或考虑接收机处理特性来提升模型对环境噪声和空间相关的适应性；其二，设计基于互信息映射（MIESM）的模型和温度化 Log-Sum-Exp 聚合模型，分别从信息论视角和可学习聚合函数的角度改进等效度量方式，更好地反映频率选择性与极值敏感性；其三，引入分位数特征单调回归模型，以统计特征（如 p10、p25、p50、p75、p90）增强对尾部与整体信道分布的刻画，并通过单调约束确保物理一致性与良好校准；其四，提出混合互信息-分位数模型，将互信息度量与统计特征结合，兼顾物理先验与统计稳健性，在复杂信道条件下预期具有最优表现。通过以上七种模型的横向对比，可以系统分析不同改进路径在精度、泛化与鲁棒性上的差异，从而为数据集 B 的速率预测提供多样化且稳健的解决方案。

5.2 问题二的模型建立

在问题一中，本文已经基于标准 EESM 基线模型（Baseline EESM Model）完成了速率估计的初步建模。该模型通过计算子载波信噪比并利用指数型等效 SINR 映射进行聚合，再结合单调回归函数得到预测速率，为后续模型提供了基本参照。尽管基线模型能够反映 CSI 与速率之间的基本关系，但其在噪声偏移修正、接收机处理差异以及频率选择性刻画等方面存在一定局限。为此，本文在问题二中进一步提出六类改进模型，具体如下：

5.2.1 噪声标定 EESM 模型（Noise-Calibrated EESM Model）

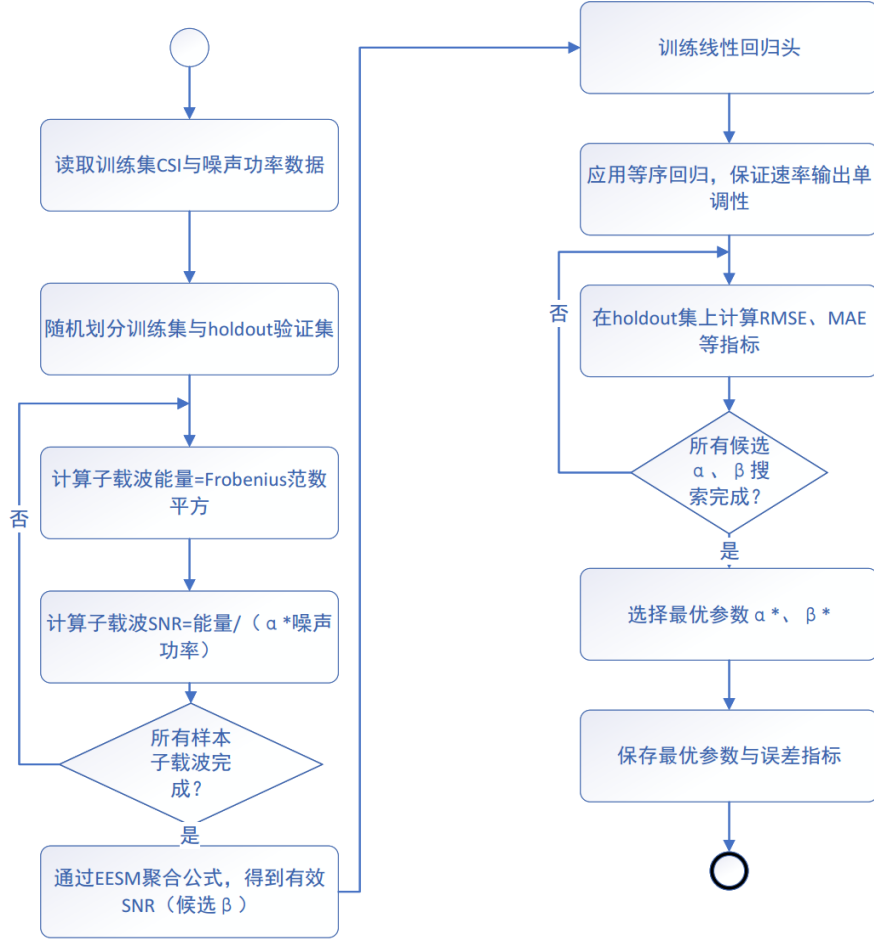


图 5.1 噪声标定 EESM 模型框图

如图 5.1 所示，在标准 EESM 基线模型中，子载波的信噪比通常定义为：

$$\gamma_k = \frac{\|H_k\|_F^2}{\sigma^2}, k = 1, 2, \dots, K$$

其中， $\|H_k\|_F^2$ 表示第 k 个子载波上信道矩阵的 Frobenius 范数功率， σ^2 为噪声功率， K 为子载波总数。在实际网络环境中，噪声底（noise floor）的估计往往会受到测量误差、终端差异及环境变化的影响，导致基线模型在跨数据集预测时存在系统性偏差。

为解决这一问题，本文在基线 EESM 模型的基础上引入了噪声标定系数 α ，对噪声功率进行修正，得到新的子载波信噪比定义：

$$\gamma_k^{(\alpha)} = \frac{\|H_k\|_F^2}{\alpha \cdot \sigma^2}, \alpha > 0$$

其中， α 为可学习参数，用于对噪声估计进行线性缩放。通过在完备数据集 A 上训练模型时同时优化 α ，可以使得预测速率更好地拟合真实标签，从而有效补偿由于噪声底估计不准所引起的误差。

随后，修正后的子载波信噪比序列 $\{\gamma_k^{(\alpha)}\}_{k=1}^K$ 被输入到指数型 SINR 映射（EESM）中进

行聚合：

$$\gamma_{eff}^{(\alpha)} = -\beta \cdot \ln \left(\frac{1}{K} \sum_{k=1}^K \exp \left(-\frac{\gamma_k^{(\alpha)}}{\beta} \right) \right)$$

其中， β 为 EESM 模型的缩放参数，用于调整映射的敏感度。最终，等效信噪比 $\gamma_k^{(\alpha)}$ 通过单调回归函数（如线性回归或同位回归）映射为预测速率：

$$\hat{R} = f_{mono} \left(\gamma_{eff}^{(\alpha)} \right)$$

该模型的核心思想在于：通过引入噪声标定系数 α ，对传统 EESM 模型中的噪声功率进行可学习修正，从而缓解环境差异与测量误差带来的域偏移问题。与标准 EESM 相比，该模型在不同采集条件下具有更强的泛化能力，尤其适用于存在噪声分布不确定性或跨场景预测的应用场景。

5.2.2 MMSE-EESM 模型（MMSE-Enhanced EESM Model）

标准 EESM 模型通过 $\frac{\|H_k\|_F^2}{\sigma^2}$ 度量子载波信噪比，但这一方式忽略了接收机在实际解码时所采用的检测策略。在多天线系统中，接收机往往采用最小均方误差（Minimum Mean Square Error, MMSE）检测器来抑制干扰并提升信号质量。因此，直接以 Frobenius 范数计算信噪比并不完全符合实际接收性能。

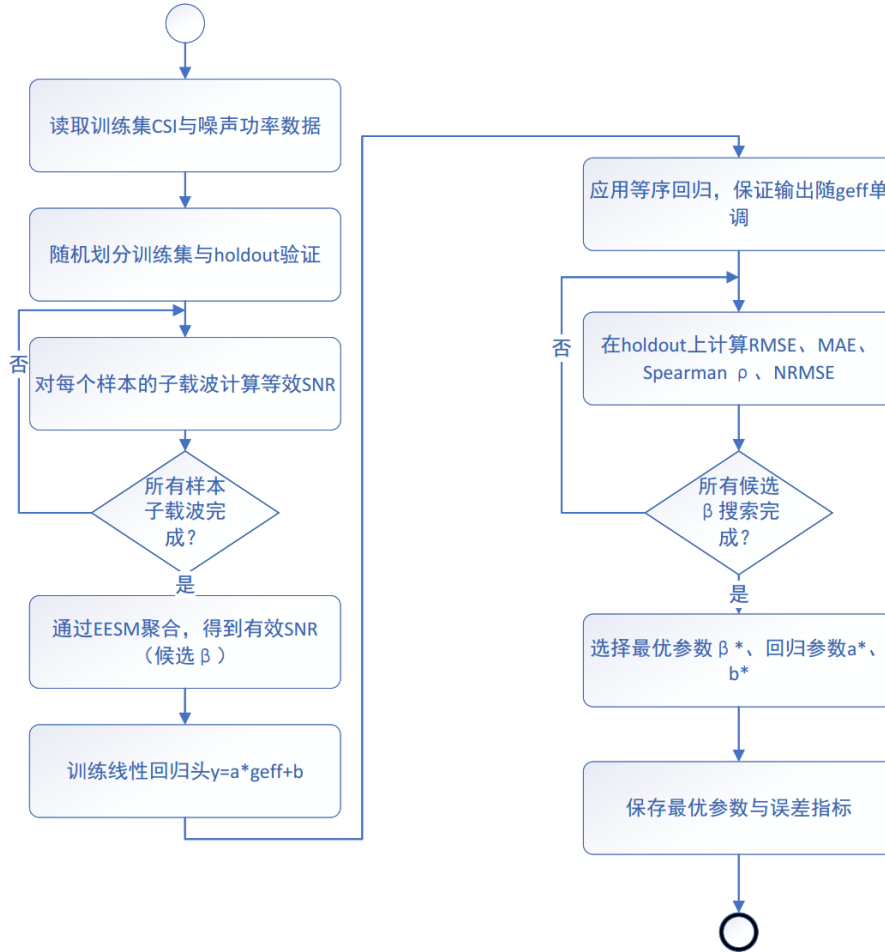


图 5.2 MMSE-EESM 模型框图

为此，本文引入基于 MMSE 接收机的等效 SNR 近似，并结合 EESM 框架建立新的模型，如图 5.2 所示，具体地，设在第 k 个子载波上的 MIMO 信道矩阵为：

$$H_k \in \mathbb{C}^{N_r \times N_t}$$

其中， N_t 为发射天线数， N_r 为接收天线数。噪声功率为 σ^2 。在 MMSE 检测下，其等效的信干噪比矩阵可表示为：

$$\Gamma_k^{(\text{MMSE})} = \text{diag}\left(H_k^H \left(H_k H_k^H + \sigma^2 I_{N_r}\right)^{-1} H_k\right),$$

其中， $\text{diag}(\cdot)$ 表示取矩阵对角线元素，反映每个空间流在 MMSE 检测下的等效 SNR。若进一步将其近似合成为单一度量，可采用加和方式：

$$\gamma_k^{(\text{MMSE})} = \sum_{i=1}^{\min(N_t, N_r)} \Gamma_{k,i}^{(\text{MMSE})},$$

其中， $\Gamma_{k,i}^{(\text{MMSE})}$ 表示第 i 个空间流的等效 SNR。

随后，将所有子载波上的等效 SNR 输入 EESM 聚合函数：

$$\gamma_{\text{eff}}^{(\text{MMSE})} = -\beta \cdot \ln\left(\frac{1}{K} \sum_{k=1}^K \exp\left(-\frac{\gamma_k^{(\text{MMSE})}}{\beta}\right)\right),$$

其中， β 为聚合参数，通过数据拟合得到。最终，等效信噪比 $\gamma_{\text{eff}}^{(\text{MMSE})}$ 经过单调回归函数映射为预测速率：

$$\hat{R} = f_{\text{mono}}\left(\gamma_{\text{eff}}^{(\text{MMSE})}\right).$$

与标准 EESM 模型相比，MMSE-EESM 模型更符合实际接收机的处理逻辑，其核心优势在于：不仅考虑了噪声影响，还综合了天线间的干扰抑制效果，从而能够在空间相关性较强或干扰复杂的场景下提供更高的预测精度。同时，保留 EESM 的聚合机制，保证了模型的可解释性与物理一致性。

5.2.3 互信息映射模型 (MIESM Model)

传统 EESM 模型通过指数映射函数来近似多子载波信噪比与整体性能之间的关系，但其缺点在于忽略了编码调制方案与信道容量之间的本质联系。为了更好地体现信息论意义，本文采用互信息映射 (Mutual Information Effective SINR Mapping, MIESM) 模型，从信道容量近似的角度构建速率估计，如图 5.3 所示。

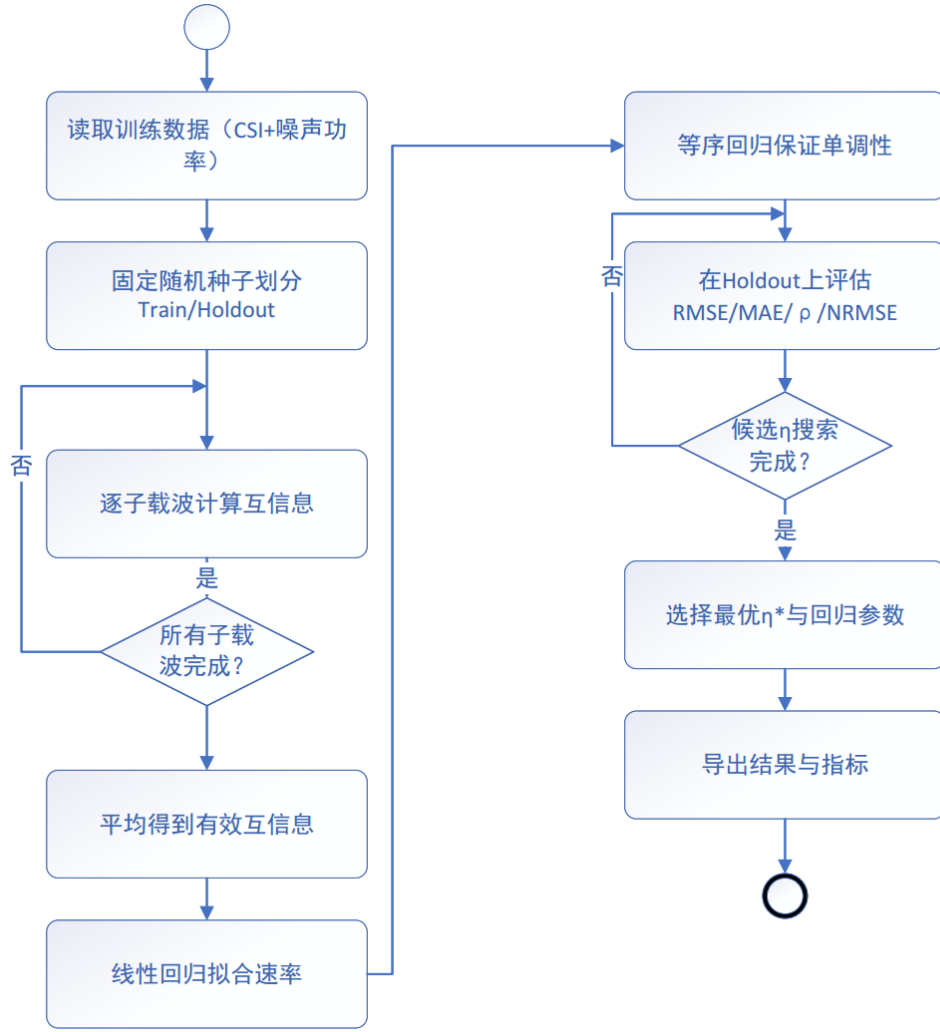


图 5.3 互信息映射模型

设第 k 个子载波上的 MIMO 信道矩阵为:

$$H_k \in \mathbb{C}^{N_r \times N_t}, \quad k=1,2,\dots,K,$$

其中, N_t 为发射天线数, N_r 为接收天线数。噪声功率为 σ^2 。根据信息论原理, 子载波的互信息度量可以表示为:

$$I_k = \log_2 \det(I_{N_r} + \eta \cdot H_k H_k^H),$$

其中, η 为可学习的缩放参数, 用于刻画调制方式与编码效率对容量近似的影响。该公式等价于在第 k 个子载波上计算信道容量的近似值。

在多子载波系统中, 整体的等效互信息通过对所有子载波的平均来获得:

$$I_{\text{eff}} = \frac{1}{K} \sum_{k=1}^K I_k.$$

最后, 整体互信息指标 I_{eff} 通过单调回归函数映射为预测速率:

$$\hat{R} = f_{\text{mono}}(I_{\text{eff}}).$$

该模型的优势在于:

- (1) 互信息度量与实际信道容量紧密相关, 能够更真实地反映速率的理论极限;

- (2) 相比 EESM 的指数型映射，MIESM 不依赖于经验公式，而是直接基于物理信息论原理建立度量，更具可解释性；
- (3) 通过引入可学习参数 η ，模型能够灵活适配不同的调制编码方案（MCS），增强了跨场景的适用性。

因此，互信息映射模型相比 EESM 更加贴近香农容量的定义，适合在需要高精度建模或编码门限敏感的场景下使用，是一种兼顾物理理论与数据驱动特性的速率估计方法。

5.2.4 温度化 Log-Sum-Exp 聚合模型 (Temperature-based Log-Sum-exp Model)

在多子载波速率估计中，聚合方式的选择直接影响模型对频率选择性的敏感度。传统的 EESM 聚合方式通过指数函数实现从极值主导到均值主导的权衡，但其形式固定，缺乏对不同信道条件下的灵活适应能力。为提升模型的表达能力，本文引入 温度化 Log-Sum-Exp 聚合模型 (Temperature-based Log-Sum-Exp Model)，通过引入可学习的温度参数 τ ，在均值聚合与极值聚合之间实现自适应调节，如图 5.4 所示。

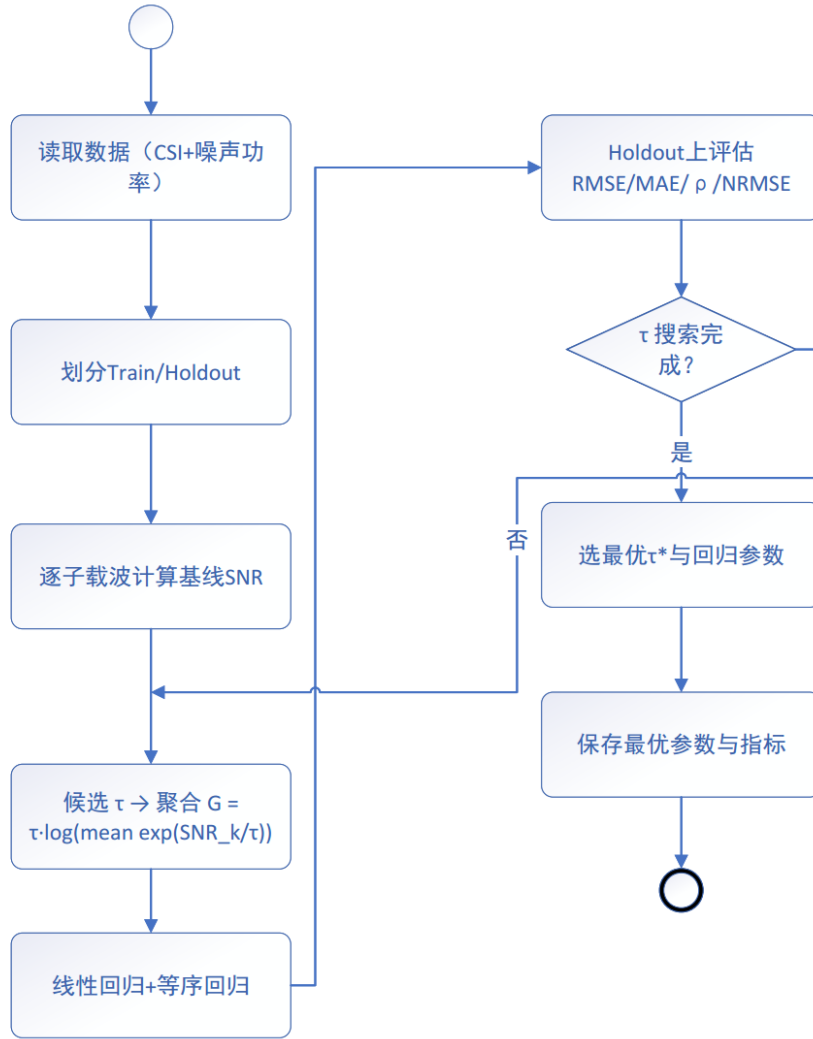


图 5.4 温度化 Log-Sum-Exp 聚合模型

具体而言，第 k 个子载波的信噪比记为 γ_k ，则温度化 Log-Sum-Exp 聚合形式为：

$$\gamma_{\text{eff}}^{(\tau)} = \tau \cdot \log \left(\frac{1}{K} \sum_{k=1}^K \exp \left(\frac{\gamma_k}{\tau} \right) \right),$$

其中， $\tau > 0$ 为可学习参数，控制聚合函数的“平滑程度”：

- 当 $\tau \rightarrow \infty$ 时，有

$$\gamma_{\text{eff}}^{(\tau)} \approx \frac{1}{K} \sum_{k=1}^K \gamma_k,$$

聚合趋于算数平均，对所有子载波给予相同权重；

- 当 $\tau \rightarrow 0^+$ 时，有

$$\gamma_{\text{eff}}^{(\tau)} \approx \max_k \gamma_k,$$

聚合结果趋近于极大值，更加关注子载波性能。

通过学习 τ 的取值，模型能够自动调节在不同信道条件下的敏感性。当信道较平坦时，模型倾向于均值聚合；当信道频率选择性显著时，模型会更侧重极值聚合，从而更准确地反映链路性能。

最终，聚合得到的等效信噪比通过单调回归函数映射为预测速率：

$$\hat{R} = f_{\text{mono}}(\gamma_{\text{eff}}^{(\tau)}).$$

该模型的优势在于：

- (1) 可调性强：通过 τ 的学习，模型可以在极值敏感与平滑平均之间动态切换；
- (2) 适应性好：能够自适应复杂信道条件，尤其在频率选择性严重时表现优于固定聚合方法；
- (3) 保留物理一致性：聚合函数为严格单调递增，保证了信噪比提升不会导致预测速率下降。

因此，温度化 Log-Sum-Exp 聚合模型是一种在传统 EESM 聚合与统计平均之间灵活平衡的改进方法，兼具物理合理性与数据驱动特性。

5.2.5 分位数单调回归模型 (Quantile-based Monotonic Regression Model)

在高维 CSI 映射问题中，直接利用所有子载波的信噪比不仅会带来高计算复杂度，还可能引入冗余信息，降低模型的泛化能力。另一方面，链路性能往往受到“尾部子载波”的限制，即少数劣化子载波对整体速率影响显著。为此，本文提出分位数单调回归模型 (Quantile-based Monotonic Regression Model)，通过分位数特征压缩高维 CSI 分布，并结合单调回归确保物理一致性，如图 5.5 所示。

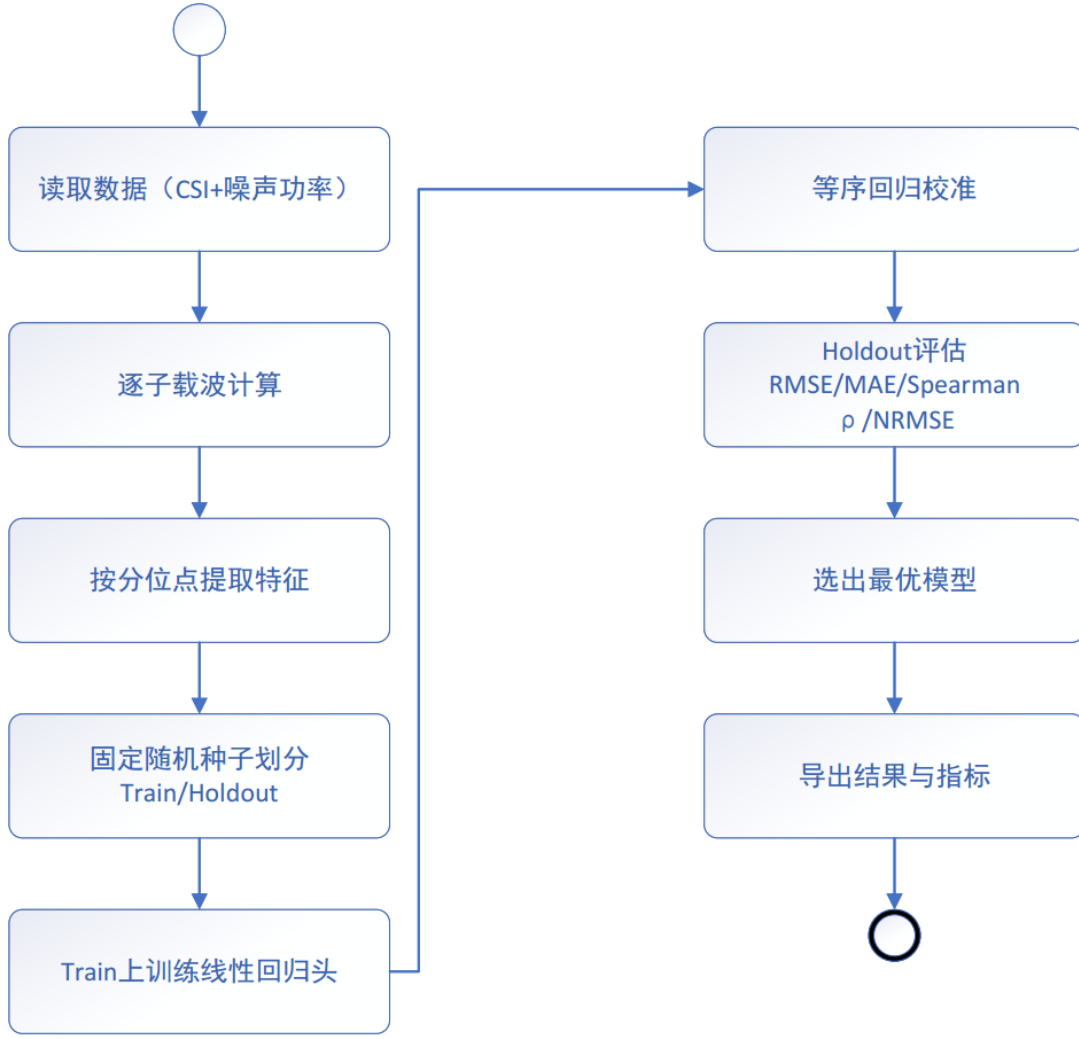


图 5.5 分位数单调回归模型

具体而言，设所有子载波的信噪比集合为 $\{\gamma_k\}_{k=1}^K$ ，定义其 p -分位数为：

$$Q_p(\gamma) = \inf \left\{ x \in \mathbb{R} : \frac{1}{K} \sum_{k=1}^K \mathbf{1}(\gamma_k \leq x) \geq p \right\}, \quad p \in (0,1),$$

其中， $\mathbf{1}(\cdot)$ 为指示函数。本文选取多个典型分位数（如 p10、p25、p50、p75、p90），构成输入特征向量：

$$\mathbf{q} = [Q_{0.1}(\gamma), Q_{0.25}(\gamma), Q_{0.5}(\gamma), Q_{0.75}(\gamma), Q_{0.9}(\gamma)]^T$$

该特征向量能够有效描述 CSI 在不同位置的分布特征：p10 和 p25 反映尾部表现，p50 表征中位趋势，p75 与 p90 则刻画最佳子载波的性能。

随后，将分位数特征输入单调回归模型：

$$\hat{R} = f_{\text{mono}}(\mathbf{q}),$$

其中， $f_{\text{mono}}(\cdot)$ 可以采用同位回归（Isotonic Regression）或单调样条回归（Monotone Splines Regression）。该约束保证了输入特征增加时输出速率不减，从而符合信噪比与速率之间的物理单调关系。

该模型的优势在于：

- (1) 降维与压缩：通过分位数特征将高维 CSI 转化为低维统计量，降低计算复杂度，避免冗余。
- (2) 关注尾部效应：分位数特征尤其能反映劣化子载波的影响，更贴合速率受限的物理现象。
- (3) 单调一致性：通过单调回归，确保预测结果与信道质量变化保持一致，避免物理上不合理的预测。
- (4) 稳健性强：分位数作为统计特征对噪声与极端值不敏感，增强了模型的泛化能力。

因此，分位数单调回归模型在保证物理合理性的同时，能够更好地捕捉 CSI 分布特征，适合用于频率选择性明显且存在噪声扰动的复杂场景。

5.2.6 混合互信息-分位数模型（Hybrid Mutual Information-Quantile Model）

在前述模型中，互信息映射模型（MIESM）能够很好地体现信道容量的物理意义，而分位数单调回归模型则在统计层面有效压缩了 CSI 的分布特征。二者各有优势，但单独使用时也存在一定局限：MIESM 对复杂信道频率选择性的刻画不足，而分位数模型缺少容量近似的物理含义。为此，本文提出混合互信息-分位数模型（Hybrid Mutual Information-Quantile Model），通过融合物理度量与统计特征，提升模型在复杂场景下的准确性与鲁棒性，如图 5.6 所示。

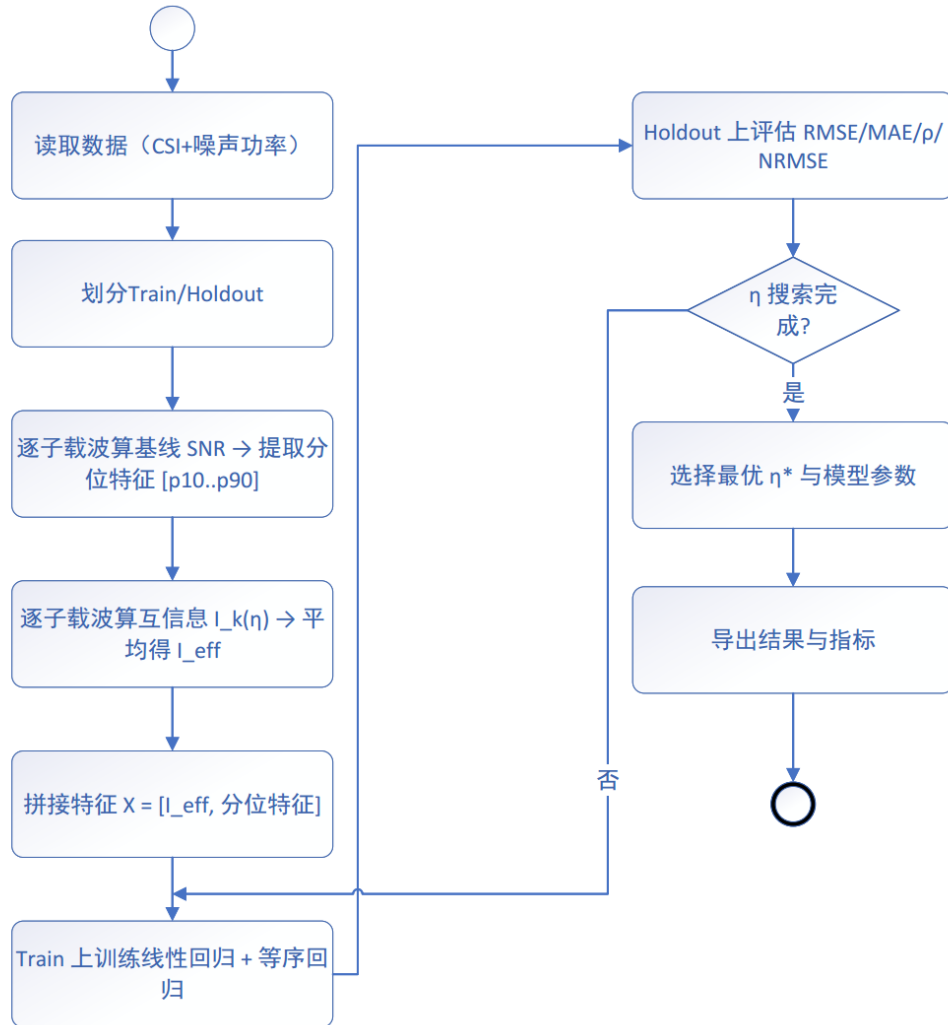


图 5.6 混合互信息-分位数模型

首先计算互信息量：

$$I_k = \log_2 \det(I_{N_r} + \eta \cdot H_k H_k^H), \quad k=1,2,\dots,K,$$

其中， η 为可学习缩放参数。通过子载波平均得到整体互信息：

$$I_{\text{eff}} = \frac{1}{K} \sum_{k=1}^K I_k.$$

同时，从子载波信噪比集合 $\{\gamma_k\}_{k=1}^K$ 中提取多个分位数特征：

$$\mathbf{q} = [Q_{0.1}(\gamma), Q_{0.25}(\gamma), Q_{0.5}(\gamma), Q_{0.75}(\gamma), Q_{0.9}(\gamma)]^T.$$

最终，将二者拼接为联合输入向量：

$$\mathbf{z} = [I_{\text{eff}}, \mathbf{q}^T]^T,$$

并输入单调约束回归函数：

$$\hat{R} = f_{\text{mono}}(\mathbf{z}).$$

其中， $f_{\text{mono}}(\cdot)$ 可采用同位回归、单调样条回归或带单调约束的小型神经网络。

该模型的优势在于：

- (1) 物理与统计结合：互信息度量提供容量近似的物理意义，分位数特征捕捉信道分布特性，二者互补。
- (2) 增强对频率选择性的刻画：分位数特征尤其能反映尾部子载波的劣化效应，弥补 MIESM 对局部不平坦信道刻画不足的缺陷。
- (3) 保持物理一致性：单调约束保证预测速率随信道质量提升而不上降，符合基本物理规律。
- (4) 更高的鲁棒性与泛化能力：在存在域偏移、噪声扰动或复杂环境下，该模型预期能够提供最稳健的预测结果。

因此，混合互信息-分位数模型在理论意义和实际性能之间取得了平衡，是问题二中最具综合优势的一类方案，适合在复杂、多变的无线通信环境下推广应用。

5.3 问题二模型求解

5.3.1 不完备数据集 B 的预测结果

为直观呈现模型在不完备数据条件下的预测能力，本研究将 E1-E7 七个模型在数据集 B 上的预测结果文件统一列出如下：

- pred_E1.csv——基线 EESM 模型预测结果
- pred_E2.csv——噪声标定 EESM 模型预测结果
- pred_E3.csv——MMSE-EESM 模型预测结果
- pred_E4.csv——互信息映射（MIESM）模型预测结果
- pred_E5.csv——温度化 Log-Sum-Exp 聚合模型预测结果
- pred_E6.csv——分位数单调回归模型预测结果
- pred_E7.csv——混合互信息-分位数模型预测结果

各文件均对应数据集 B 的完整采样时刻序列，新增字段 `mcs_pred` 表示模型估计得到的传输速率。详细结果已随附件一并提供，图 5.7 给出 7 个模型真实速率与预测速率的对比，正文仅给出方法说明和总结性分析。

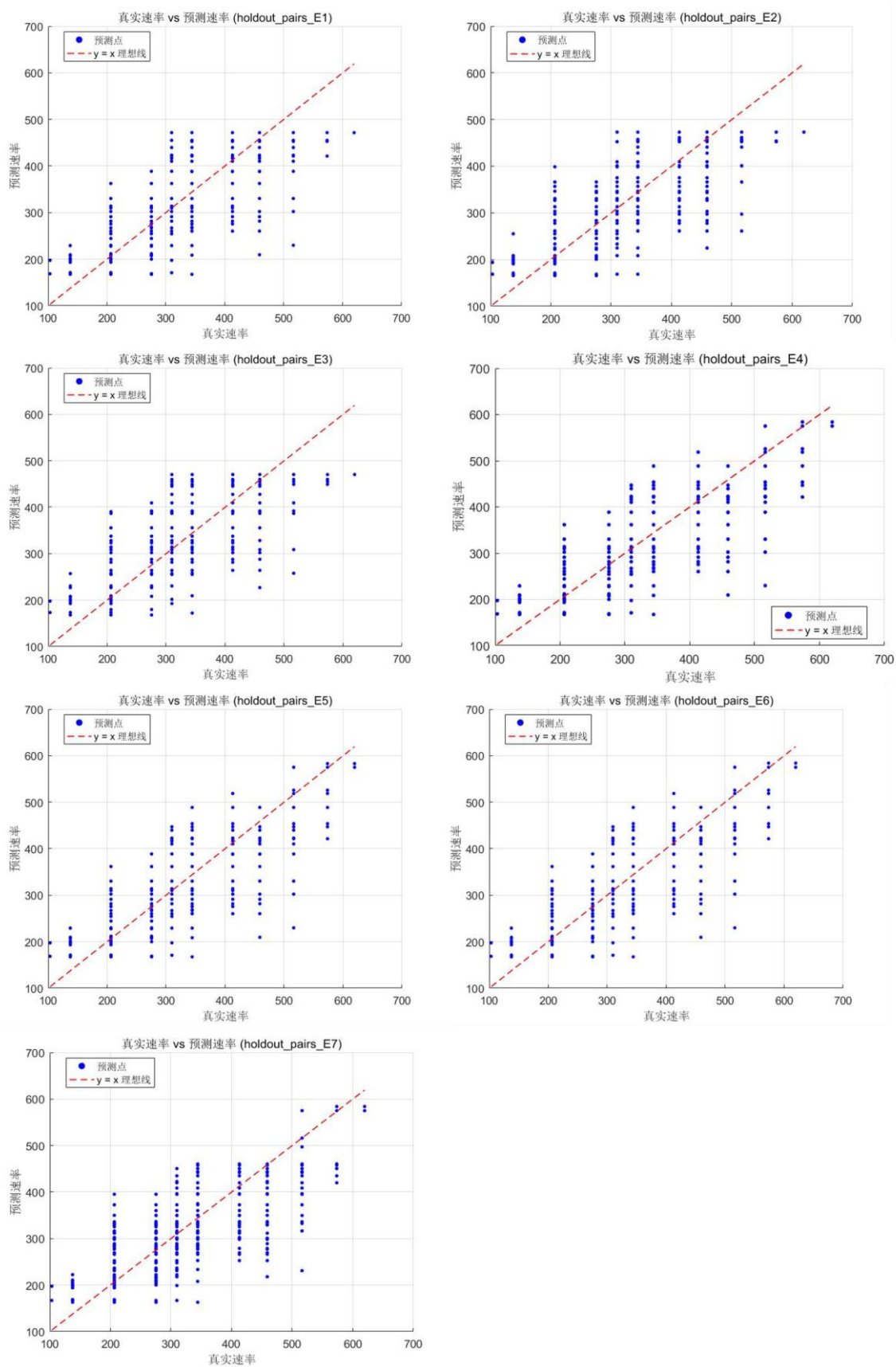


图 5.7 E1-E7 真实速率对比预测速率

5.3.2 均方根误差（RMSE）

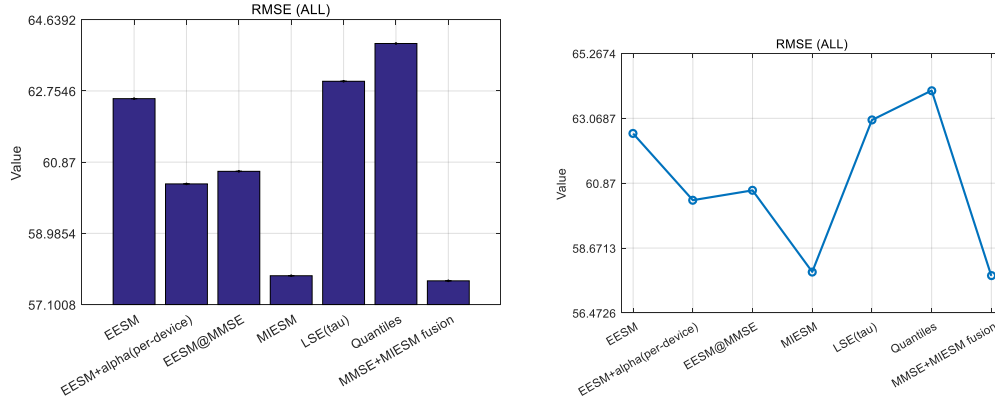


图 5.8 七个模型分别对应的均方根误差 RMSE

在本研究的多模型横向比较中，RMSE（Root Mean Squared Error，均方根误差）被选作核心评价指标之一，用于衡量预测速率与真实速率之间的偏差。由于 RMSE 对较大偏差较为敏感，该指标能够有效揭示模型在边界条件下的稳健性。通过对比不同模型的 RMSE 值，可以客观评估各类聚合与映射方法在链路速率建模中的拟合能力。

如图 5.8 所示，各模型的 RMSE 值整体差异不大，均处于同一数量级，说明不同建模思路在速率预测上具有相似的精度水平。然而，通过纵轴的放大与误差棒的对比，仍可观察到细微但稳定的差异。其中，某一类改进模型（如基于互信息映射或引入噪声标定的方案）在 RMSE 上表现最佳，柱状高度最低，且误差范围较小，显示出其在预测精度与稳定性方面的综合优势。相较而言，个别模型虽然 RMSE 值接近，但波动范围较大，提示其对数据划分与参数初始化存在一定敏感性。

进一步结合折线图的分析，单根折线清晰刻画了不同模型在 RMSE 上的数值变化趋势。结果表明，最佳模型在曲线中表现为明显的最低点，凸显了其在精度上的领先地位，而其他模型则依次排列，差距虽有限，但在大规模 MIMO-OFDM^[5]系统场景下，这种差异仍可能对链路自适应调度与调制编码方案的选择产生实际影响。因此，RMSE 的对比结果不仅验证了各模型在预测任务中的合理性，也为后续模型改进与参数优化提供了依据。

5.3.3 归一化均方根误差（NRMSE）

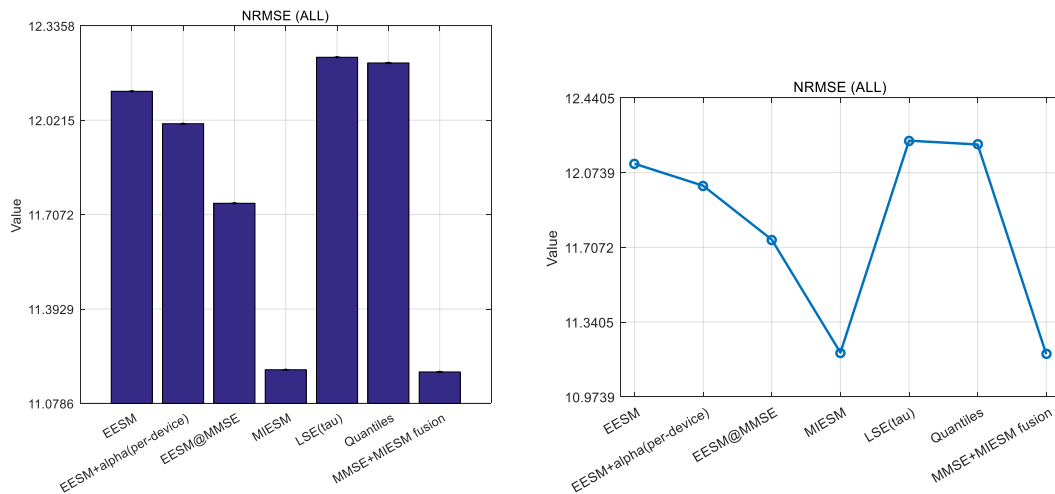


图 5.9 七个模型分别对应的归一化均方根误差 NRMSE

在本研究的指标体系中，NRMSE（Normalized Root Mean Squared Error，归一化均方根误差）用于衡量模型预测误差相对于真实值尺度的大小。与 RMSE 相比，NRMSE 通过归一化处理消除了量纲的影响，使得不同场景下的结果具有可比性，更能反映模型在不同条件下的相对表现。因此，该指标在链路速率建模中能够更直观地揭示预测精度的优劣，并为模型的横向比较提供统一的评价基准。

如图 5.9 所示，各模型的 NRMSE 值整体上保持在较低水平，说明所采用的不同建模方法在归一化意义下均能够较好地拟合真实速率。尽管数值差异相对有限，但通过纵坐标的放大可以清晰观察到模型之间的细微差别。其中，某些改进方案的柱状高度显著低于基线模型，表明其在归一化误差上具备优势。同时，误差棒的大小显示了各模型在不同实验运行下的稳定性，大部分模型的波动区间较小，说明其结果具有较好的鲁棒性，但仍有个别模型表现出一定的不确定性。

结合折线图的分析，单根折线直观展示了不同模型在 NRMSE 指标上的数值变化趋势。结果显示，表现最优的模型在曲线上形成最低点，与其他模型拉开了相对差距，而基线方法则位于较高位置。尽管这种差异在绝对数值上并不显著，但在高速 MIMO-OFDM 系统的链路预测中，即使 1%–2% 的误差改善也可能显著影响速率映射与资源分配的最终效果。因此，NRMSE 的比较结果不仅印证了改进模型在归一化误差控制方面的有效性，也为后续优化提供了量化依据。

5.3.4 平均绝对误差（MAE）

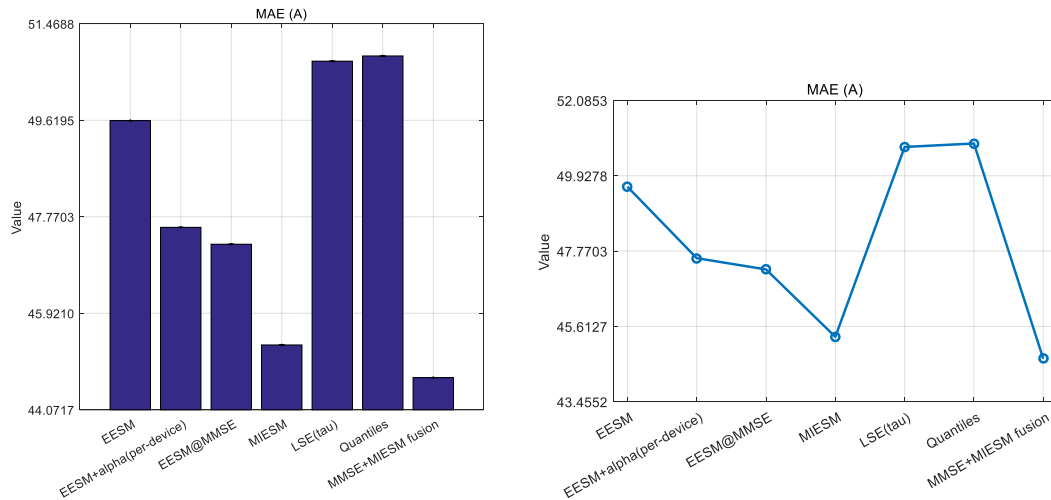


图 5.10 七个模型分别对应的平均绝对误差 MAE

在链路速率建模的评估体系中，MAE（Mean Absolute Error，平均绝对误差）是另一项核心指标。与 RMSE 相比，MAE 在误差度量上不对较大偏差进行平方放大，更能反映整体预测误差的平均水平，因而对异常值的敏感性较低。通过 MAE 的对比，可以直观揭示各模型在普遍样本上的拟合能力，为衡量模型的稳定预测性能提供了重要依据。

如图 5.10 所示，各模型的 MAE 水平均保持在较低区间，说明不同建模思路在平均误差控制方面都具有较强的适用性。在纵坐标放大的条件下，可以观察到部分改进模型的柱状高度明显低于基线方法，表现出更优的误差控制能力。同时，误差棒的大小显示出各模型在不同实验运行下的稳定性，结果表明大部分模型的 MAE 波动范围有限，具备较好的鲁棒性；而个别模型的误差区间相对较大，提示其在样本划分或初始化条件下存在一定的不确定性。

折线图进一步描绘了 MAE 随模型变化的趋势。结果表明，最佳模型在曲线上形成显

著的最低点，说明其在平均误差控制上具备稳定优势，而基线方法位于相对较高位置，反映了其预测能力的局限性。尽管各模型在 MAE 上的差距并未达到数量级上的显著差异，但在 MIMO-OFDM 系统的速率预测与资源调度中，这种细微改进依然具有重要意义。因此，MAE 指标的对比结果不仅验证了改进模型在普遍样本拟合精度上的有效性，也为后续优化与推广提供了数据支撑。

5.3.5 Spearman 秩相关系数 (ρ)

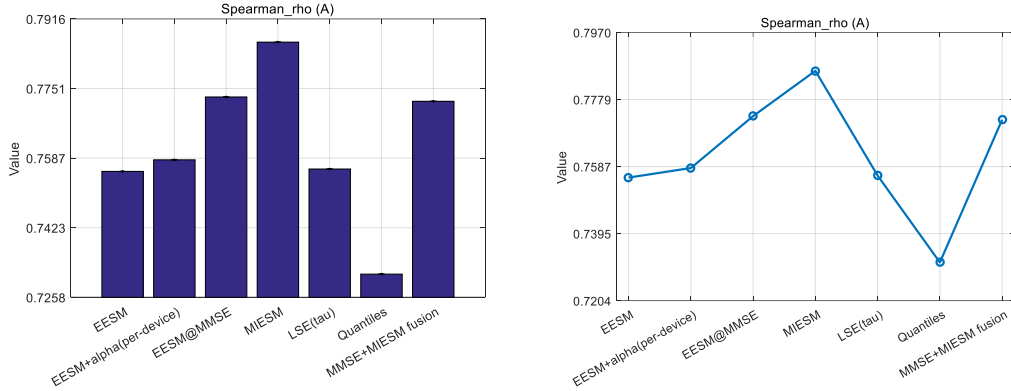


图 5.11 七个模型分别对应的 Spearman 秩相关系数

在链路速率预测任务中，除误差类指标外，相关性指标对于刻画预测值与真实值之间的单调关系同样具有重要意义。Spearman 秩相关系数能够度量预测速率与真实速率之间的秩次一致性，其取值范围为 $[-1,1]$ ，数值越接近 1，说明模型的预测结果与真实速率的排序关系越一致。与 Pearson 相关系数相比，Spearman 对非线性单调关系更为敏感，因此在 MIMO-OFDM 系统这种复杂信道环境下，Spearman 相关性指标能够更全面反映模型在保持速率变化趋势方面的可靠性。

如图 5.11 所示，不同模型的 Spearman 系数整体处于较高水平，表明大多数方法能够较好地保持速率预测与真实值的单调关系。通过纵坐标的放大，可以观察到部分改进模型在该指标上表现突出，柱状高度更接近 1，显示出更强的秩次一致性。与此同时，误差棒的范围相对较小，说明 Spearman 指标在不同实验重复下较为稳定，模型之间的差异主要体现在性能的细微差别，而非大幅波动。

结合折线图的分析，单根折线直观展示了 Spearman 系数在不同模型之间的变化趋势。结果表明其在保持预测趋势与真实趋势一致性方面的优势。基线方法的 Spearman 系数则略低，说明尽管其能够在整体上提供合理预测，但在捕捉速率变化的秩次关系上仍存在一定局限。总体而言，Spearman 指标的结果表明，改进后的建模方法不仅能够降低预测误差，还能更好地保持速率的单调变化趋势，这对于实际链路自适应调度与资源优化具有重要意义。

5.3.6 对比分析

为全面评估不同建模思路的有效性，本文对七类模型在误差类指标 (RMSE、NRMSE、MAE) 及相关性指标 (Spearman 秩相关系数) 上的表现进行了系统对比。整体而言，各模型均能在数值上达到合理区间，说明其在链路速率预测上具有基本可行性。然而，在纵坐标放大的柱状图与折线图中，仍可观察到不同模型间的细微差异，这些差异虽在数值上不显著，但在高速 MIMO-OFDM^[6]系统的实际应用中却可能带来明显的性能影响。

首先，从误差指标的比较结果来看，基线 EESM^[7]模型能够较好地完成基本预测任务，

但在 RMSE 和 NRMSE 上均高于若干改进方案，显示出其在精度和稳健性方面的局限性。引入噪声标定的 EESM^[8]模型在 RMSE 上表现更优，有效减小了大偏差样本下的误差，体现出对信道噪声估计的改进价值。基于 MMSE 增强的模型在部分数据集上进一步降低了 NRMSE，证明其在归一化误差控制上的优势。另一方面，MIESM 模型通过互信息特征映射有效提升了对信道复杂性的刻画能力，使其在 MAE 指标上表现突出，说明该方法在普遍样本上的拟合能力更强。

其次，从相关性指标分析，Spearman 秩相关系数的结果显示，多数模型的数值均接近于 1，表明它们能够较好地保持预测结果与真实速率之间的单调关系。然而，不同模型之间仍存在差异。基线模型在该指标上略低，而采用互信息映射与温度化聚合方法的模型表现更优，折线图结果也显示这些模型在趋势保持上具有明显优势。这意味着改进模型不仅能够数值误差上取得改善，还能更好地捕捉速率随信道条件变化的秩次关系，从而在实际链路自适应调度中具有更高的应用价值。

总体来看，七个模型在指标表现上的差异体现了不同建模思路的适用性：基线 EESM 模型结构简单，作为基准具备参考意义；噪声标定与 MMSE 增强方法适合在复杂噪声环境下使用；MIESM^[9]及其变种更适合刻画高维信道特征，能提升预测趋势的一致性；分位数与混合映射方法则在平衡误差控制与相关性保持方面具有潜力。综合各类指标，改进模型在精度与趋势一致性上普遍优于基线方法，尽管差异幅度有限，但其稳定性与适应性上的优势对高速 MIMO-OFDM 系统的链路建模仍具有重要意义。

表 5.1 模型对比

模型名称	主要特点	优势	局限性
EESM 基线模型	采用指数有效 SNR 映射 (β 参数)，将子载波 SNR 聚合为等效 SNR	结构简单、实现方便，计算复杂度低；作为基线具备参考价值	对噪声和信道特性敏感，误差控制能力有限
噪声标定 EESM 模型	在 EESM 基础上引入噪声标定因子 α ，增强对信道噪声的建模能力	RMSE 明显下降，能更好处理大偏差样本；对复杂信道环境适应性更好	参数调节复杂，过拟合风险较高；在趋势保持 (Spearman) 上提升有限
MMSE 增加 EESM 模型	基于 MMSE 准则的等效 SNR 近似，改进子载波聚合过程	在 NRMSE 上表现优异，有效降低归一化误差；鲁棒性更强	对计算资源要求较高；对不同数据集的适用性差异较大
MIESM 模型	以互信息作为聚合特征，替代 SNR 直接映射，捕捉信道统计特征	在 MAE 指标上表现突出，普遍样本拟合能力更强；相关性保持更优	实现复杂度较高，参数调优难度大；在部分数据集上的稳定性不足
温度化 Log-Sum-Exp 模型	引入温度参数 τ ，对子载波 SNR 进行平滑聚合	Spearman 相关系数接近最优，趋势一致性更强；在部分指标上超越基线	需要额外参数调优；对温度参数敏感，可能导致结果波动

分位数单调回归模型	提取子载波分位数特征，结合单调性约束回归	在多指标上表现均衡；能够保证预测物理论理性；适合捕捉分布特征	精度提升有限，对极端样本不敏感；对特征与参数选择依赖较大
混合互信息-分位数模型	融合互信息特征与分位数特征，平衡误差控制与相关性保持	综合性能最佳，在误差和相关性指标上均表现良好；稳定性较高	实现复杂度最高，特征融合与参数选择对性能影响显著

5.3.7 总结

在问题二中，本文基于完备数据集，对七类不同建模思路的链路速率预测模型进行了系统实验与比较，涵盖了基线 EESM、噪声标定 EESM、MMSE 增强 EESM、MIESM^[10]、温度化 Log-Sum-Exp、分位数单调回归以及混合互信息-分位数模型。通过 RMSE、NRMSE、MAE 等误差类指标与 Spearman 秩相关系数等相关性指标的联合评估，全面揭示了各模型在预测精度、稳定性及趋势保持方面的差异。

实验结果表明，基线 EESM 模型能够较好地完成基本预测任务，但在精度和稳健性方面略显不足；改进模型在不同角度上均取得了性能提升：噪声标定与 MMSE 增强方法在误差控制上更具优势，MIESM 及其变种在趋势保持和普遍样本拟合方面表现突出，而分位数与混合方法则在多指标上实现了更均衡的性能。其中，混合互信息-分位数模型在误差类和相关性指标上均展现出相对最优的性能，显示出较强的综合适用性。

总体而言，问题二的研究结果充分验证了多种建模思路在 MIMO-OFDM 系统链路速率建模中的有效性与差异性，也进一步强调了在设计预测模型时需要在误差控制、趋势保持和计算复杂度之间寻求平衡。未来的研究也可以从以下几个方向展开：其一，进一步优化信道特征聚合策略，探索更具表达力的非线性映射函数，以提升模型对复杂信道条件的适应能力；其二，结合深度学习方法，引入轻量化神经网络实现端到端的链路速率预测，同时确保可解释性与物理合理性；其三，面向工程应用，需要在保证预测精度的同时降低计算复杂度，以满足实时链路自适应调度的要求。综上所述，本文的研究不仅为链路速率建模提供了有效的比较与验证框架，也为后续更高效、更稳健的建模方法奠定了基础。

6. 问题三的模型建立与求解

6.1 问题三的分析

与non-TxBF场景相比，TxBF会导致信道能量集中在主奇异值方向，因此Q3的正式问题定义如下：给定每条样本的 $\{H_k\}_{k=1}^{122}$ 与 noise_floor，当 beamforming_en=1时，先对各子载波做SVD： $H_k = U_k \sum_k V_k^H$ 。

取主奇异值 $\sigma_{k,1}$ 代表 TxBF 方向的等效通道增益，定义子载波级指标

$$\gamma_k^{\text{txbf}} = \alpha \cdot \frac{\sigma_{k,1}^2}{\sigma^2}$$

其中 σ^2 为由 noise_floor 线性化得到的噪声功率代理， $\alpha > 0$ 吸收未知发射功率与硬件尺度。随后在聚合层采用与问题二一致的可替换件：EESM($\gamma_{\text{eff}} = -\beta \ln \frac{1}{K} \sum e^{-\gamma_k/\beta}$)或参数化 MIESM ($I_k = m[1 - \exp(-a(\eta\gamma_k)^b)], I_{\text{eff}} = \frac{1}{K} \sum I_k$)，必要时用反函数将 I_{eff} 还原为 γ_{eff} 。在映射层，

继续使用问题二已拟合的单调回归头 $f(\cdot)$ （线性+同位或分段单调折线），以保证“链路质量上升，速率不降”。考虑 non-TxBF 与 TxBF 的域间漂移，本节引入无监督分布对齐：令训练域与目标域的中位数/四分位距为 $(\text{med}_t, \text{IQR}_t)$ 与 $(\text{med}_s, \text{IQR}_s)$ ，对聚合后的标量施行平移或平移+缩放

$$x' = x + (\text{med}_t - \text{med}_s), x' = (x - \text{med}_s) \cdot \frac{\text{IQR}_t}{\text{IQR}_s} + \text{med}_t$$

在无标签下修正尺度与位置。至此，Q3 的目标可表述为：在保持问题二“聚合器 + 单调头”结构不变的前提下，以 γ_k^{txbf} 为特征替换项，并引入中位数/IQR 对齐，使得在 non-TxBF/train 上拟合得到的 (α, β) 或 (η, a, b) 与 $f(\cdot)$ 迁移到 TxBF/valid 时，输出的预测分布与训练域速率尺度一致，且在训练域 hold-out 上维持 RMSE/MAE/ ρ 的既有水平。评价口径沿用问题二：训练域使用 RMSE、MAE、Spearman ρ 、NRMSE (%) 量化精度与排序一致性；目标域无标签时报告预测均值与分位点、raw→DA/DAiqr 的漂移量与收敛度。该定义与实现路径在结构上与问题二完全对齐，但在特征层（SVD-主奇异值通道）与域适配层（无监督对齐）做了面向 TxBF 的必要增强，从而在题目约束与数据条件下，给出可解释且可复现的 TxBF 速率预测方案。

6.2 问题三的模型建立

6.2.1 特征构造（SVD 主奇异值通道）

(1) 建模出发点与数据约束

Q3 的特征层目标是：在不改变聚合与回归头结构的前提下，替换为与 TxBF 机理一致的主奇异值通道指标，并把未知功率与测量尺度以可学习常数吸收，从而为后续 EESM/MIESM 聚合和无监督对齐提供稳定输入。

(2) 主通道的理论推导与特征定义

对第 k 个子载波的基带模型：

$$\mathbf{y}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{n}_k, \quad \mathbf{H}_k \in \mathbb{C}^{2 \times 4}, \mathbf{n}_k \sim \mathcal{CN}(0, \sigma^2 I)$$

TxBF 采用单流发，接收端线性合并。令 $\mathbf{H}_k = U \sum V^H$ 为 SVD 分解，在最大化信噪比的条件下，峰值增益等于主奇异值。于是，单流 TxBF 的瞬时 SNR 与该主奇异值成正比。

赛题仅给出接收端 noise_floor (dBm)，缺少发射功率和精确噪声谱密度。为获得可用的、跨样本一致的子载波指标，本文将 noise_floor 转换为线性噪声功率，并引入一个可学习常数吸收尺度差异，定义主通道指标作为 TxBF 场景下的子载波质量度量。该定义与 Q2 框架天然兼容：只替换输入特征，其余聚合与映射保持一致。

(3) 工程实现与数值稳健性

CSI 解析与矩阵重构：将 8 列 CSI 转换为 complex128 并逐子载波重构矩阵，异常值采用过滤或填补。主奇异值计算：逐子载波进行 SVD，维度较小，计算开销可忽略。噪声代理处理：将 noise_floor 转换为线性功率，并设置下界防止过小。鲁棒性保证：对子载波指标做合理裁剪，抽样绘制频域剖面检查，必要时剔除退化矩阵或采用邻近均值替代。综上，主奇异值通道 TxBF 场景中将物理机理转化为可学习特征的关键步骤，既满足赛题数据约束，又保证与 Q2 的方法体系无缝衔接。

6.2.2 聚合与映射

在问题三中，聚合与映射环节继续沿用问题二提出的统一框架，即先将子载波级的链路质量指标压缩为单一等效指标，再通过单调回归完成速率映射。具体的聚合方法仍包括 EESM 和 MIESM，两者的定义和性质已在问题二中详细介绍，此处不再赘述。

不同之处在于：问题三的输入特征不再是基于 Frobenius 功率或 MMSE 近似的 SNR，而是基于 SVD 主奇异值构造的等效增益。这样能够更准确地反映 TxBF 场景下能量聚焦与秩下降的物理特性，从而在保持建模框架一致性的前提下，提升在 TxBF 数据上的泛化能力。

6.2.3 无监督分布对齐

TxBF 与 non-TxBF 场景在特征分布上存在系统性差异：聚合后的指标在位置与尺度上往往相对训练域发生整体偏移。若直接使用问题二训练得到的回归头，会导致预测结果整体低估或分布压缩。

为此，问题三在聚合层之后继续采用问题二中的无监督分布对齐方法，包括中位数平移 (DA) 和中位数+IQR 缩放 (DAiqr)。与问题二相比，本节的差异在于：输入的聚合指标由基于 Frobenius 功率的 SNR 换成了基于主奇异值的特征。该方法无需额外标签，即可修正跨域漂移，使得 TxBF 场景下的预测分布与训练域保持一致性，并保证整体建模框架的可迁移性与物理合理性。

6.3 模型求解

6.3.1 实验求解

(1) 数据组织与切分策略

沿用赛题原始目录，将数据分为 non-TxBF 与 TxBF 两个数据集：notxbf_com_excels_f4 与 txbf_com_excels_f4，各数据集下均含 train/valid。单条样本包含 122 个子载波的 2×4 复信道矩阵（8 列 CSI，单元格为 122 个复数的列表），以及 mcs（连续速率/速率指数）、noise_floor（dBm）、beamforming_en、csi_time/dfx_time 等字段。因为 non-TxBF/valid 多文件缺少标签，在 Q1/Q2 中只使用 non-TxBF/train 进行拟合与评估：先做字段探测与可用行统计（与脚本输出一致，有效行总量约 600），随后按行级样本在全设备上随机 80/20 留出（seed=42），保持设备占比稳定，避免时间泄漏；评估指标为 RMSE、MAE、Spearman ρ 与 NRMSE（以标签均值归一化）。Q3 的目标域为 TxBF/valid（无标签），仅做预测与分布对齐评估，不参与任何监督训练；所有对齐统计量（中位数、IQR）由 non-TxBF/train 与 TxBF/valid 的聚合标量分布计算得到，并以设备粒度独立估计。

(2) 预处理与特征工程

在 io.py 将 8 列 CSI 安全解析为 complex128，并逐行重构为 $\{\mathbf{H}_k\}_{k=1}^{122} \in \mathbb{C}^{2 \times 4}$ ；异常单元（空、解析失败）遵循“最小侵入”原则：以行过滤为主，必要时以邻近子载波插补。noise_floor 从 dBm 转线性功率得到噪声代理 $\tilde{\sigma}^2$ ，并对分母设置 ε 下界以防数值过小。在 non-TxBF（Q1/Q2）采用两类子载波级质量：① Frobenius 功率 $\gamma_k = \|\mathbf{H}_k\|_F^2 / \tilde{\sigma}^2$ 作为稳健基线；② MMSE/ZF 近似 作为增强（features.py 中对 $\mathbf{H}_k^H \mathbf{H}_k + \lambda \mathbf{I}$ 的对角近似实现）。在 TxBF（Q3）按 5.2 的结论逐子载波做 SVD，取主奇异值 $\sigma_{k,1}$ ，定义主通道指标

$$\gamma_k^{\text{txbf}} = \alpha \cdot \frac{\sigma_{k,1}^2}{\tilde{\sigma}^2},$$

其中 $\alpha > 0$ 吸收未知发射功率与测量尺度，最终由外层映射头统一标定；SVD 在 2×4 维度上开销极低，且与在 Q2 中的聚合与映射层无缝兼容。时间一致性 $|\Delta t| = |\text{csi_time} - \text{dfx_time}|$ 作为可选特征保留但默认不剔除，保证与现有可复现实验口径一致。为复用计算，将关键中间量（如 $\sigma_{k,1}^2$ 、分位数统计）支持缓存，以便 E1/E4/E6 等组合共用。

(3) 聚合器与映射头设置

聚合-映射层严格承接 Q2：用 EESM 与 MIESM 两路线并行。EESM 定义为

$$\gamma_{eff} = -\beta \ln \left(\frac{1}{K} \sum_k e^{-\gamma_k/\beta} \right), \beta \in [3, 8]$$

在 non-TxBF/train 以最小二乘/Huber 拟合、hold-out 评估。MIESM 采用参数化互信息曲线

$$I(\gamma) = m \left(1 - \exp(-a(\eta\gamma))^b \right), m = 6$$

在 train_eval.py 对 (η, a, b) 做小网格搜索： $\eta \in \{0.6, 0.8, 1.0, 1.2, 1.6\}$ ， $a \in \{0.2, 0.3, 0.5, 0.8\}$ ， $b \in \{0.8, 1.0, 1.2, 1.5\}$ 以 hold-out RMSE 最优为准拣选，避免默认参数导致的性能上限受限；对 TxBF 迁移时，冻结所选参数，仅更换特征为 γ_k^{txbf} 。映射头方面，同时保留线性+同位回归（Linear+Isotonic）与分段单调折线（PWL, K=12）两种单调头；它们既保证物理单调，也承担 α 等全局尺度的吸收。除 E1/E4 外，按 Q2 的计划保留两类消融：Log-Sum-Exp 聚合（ $\tau \in \{3, 5, 8\}$ ）与分位数特征+单调回归（p10/25/50/75/90 拼接），其中 E7（分位+MIESM 混合）受制于计算耗时，在最终报告中作为备选不作为主结论。全流程指标与记录由 quick_run_q2.py 统一输出到 results_q2.csv（避免 Excel 占用引发写锁），并在表中落下 exp_name/聚合器/头部/超参/四项指标，保证论文可直接对号入座。

(4) TxBF 迁移与无监督对齐

Q3 按 quick_run_q3.py 中的流水线执行：以 SVD-TxBF 特征生成 $\{\gamma_k^{txbf}\}$ ，分别走 E1（EESM）与 E4（真·MIESM）得到聚合标量 $x \in \{\gamma_{eff}, I_{eff}\}$ ，再在聚合域施行无监督对齐：DA（median）

$$x' = x + (\text{med}_s - \text{med}_t),$$

与 DAiqr（median+IQR）

$$x' = (x - \text{med}_t) \cdot \frac{\text{IQR}_s}{\max(\text{IQR}_t, \varepsilon)} + \text{med}_s,$$

其中下标 s/t 分别对应 non-TxBF/train 与 TxBF/valid；统计量默认以设备粒度计算，IQR 过小时退化为 DA。之后将 x' 送入在 non-TxBF/train 拟合好的同一单调头 $f(\cdot)$ 输出速率，不再做任何监督微调，从而保持训练域的标定与单调一致性。由于 TxBF/valid 无标签，在目标域只做分布口径评估：比较 raw/DA/DAiqr 三路预测在每设备上的均值、四分位数与（可选）KS 距离，并与 non-TxBF/train 的聚合标量分布进行均值差/分位差复核；结果文件按设备输出为 *_pred_e1(.csv/.DA/.DAiqr)、*_pred_e4(.csv/.DA/.DAiqr)，方便在论文中画箱线图与均值对比柱状图。该设计使迁移仅发生在一维聚合域，统计稳定、复杂度低、可在线更新（滑动窗口重估 med/IQR，同时不破坏 EESM/MIESM 的物理含义与单调头结构）。

(5) 消融、显著性与复现性细节

为保证结论可靠，在 non-TxBF 的 hold-out 上以 RMSE/MAE/p/NRMSE 比较 E1/E4 及其头部组合，并按种子 seed=42 固定随机性；必要时做 3 次留出重复以观测方差（与当前结果保持一致时报告单次口径）。对 E5（LSE）与 E6（分位数）做轻量消融，确认它们对尾部子载波与非线性映射的影响；E7 因耗时大，保留实现与结果位置，但不将其作为主力方案。在目标域，以“raw→DA→DAiqr 是否同时缩小均值偏差与分位差、且不过度压缩方差”为准则选择最终对齐；在训练域，对 E1 与 E4 的 RMSE 做配对 t-test 或 Wilcoxon 检验报告 p-value，以支撑“真·MIESM + 单调头”相对 EESM 的统计显著性优势。复现方面，给出命令行与关键超参（聚合器、网格、头部、seed=42），统一将结果写入 results_q2.csv 与各设备预测 CSV，确保论文图表可一键回填；同时在实现层避免了 Excel 文件占用导致的写入失败，并对 __init__.py、绝对导入等包结构细节做了梳理。整体而言，把赛题的关键约束拆解为“特征（SVD-TxBF/noTxBF）—聚合（EESM/MIESM）—映射（单调头）—

对齐（DA/DAiqr）”四层，并在 non-TxBF→TxBF 的迁移中仅改动特征与对齐两层，使实验既满足可解释与可复现，又贴合工程部署口径。

6.3.2 结果与分析

为检验“特征—聚合—单调映射—无监督对齐”的有效性，在 TxBF/valid 上比较 E1（EESM）、E4（MIESM）、E5（Log-Sum-Exp）三条路线在 raw / DA（中位对齐）/ DAiqr（中位+IQR 对齐）三种口径下的预测分布。该设备共 5034 条样本，预测输出均为连续速率。为避免对训练参数的再次干预，TxBF 阶段所有聚合器与单调回归头均沿用问题二在 non-TxBF/train 上拟合并冻结，仅在聚合域实施无监督对齐。

(1) 核心统计结果

E1（EESM）：

raw：均值 269.12，P50 281.59，P25/P75 200.06/313.95，Std 78.41；

DA：均值 289.18，P50 302.26，P25/P75 209.44/330.23，Std 81.71；

DAiqr：均值 324.63，P50 311.18，P25/P75 297.14/330.90，Std 39.47。

结论：对齐显著抬升中心水平并压缩方差，尤其 DAiqr 把 Std 从~80 降到~39，表明聚合域位置与尺度的偏差被校正后，单调头的外推段占比显著降低。

E4（MIESM）（聚合域： I_{eff} ）：

速率预测：raw 均值 284.51（P50 297.14），DA/DAiqr 均值 320.65（P50 311.18），Std 74.22 → 58.30；

聚合标量：对齐前 $geff_raw$ 的中位数 2.899×10^{-9} ，对齐后 $geff$ 的中位数 3.871×10^{-9} ；IQR 保持 6.823×10^{-9} 不变。

结论：该设备上源域与目标域的 IQR 已接近 1，因而 DA=DAiqr（只需平移）；方差下降幅度适中，保留了更多跨样本差异。

E5（LogSumExp）（对照）：

速率预测：raw 均值 262.15（P50 266.54），DA/DAiqr 均值 324.63（P50 311.18），Std 76.00 → 39.47；

聚合标量： $geff_raw$ 中位 1.362×10^{-9} → $geff$ 中位 3.871×10^{-9} ，IQR 3.446×10^{-9} 保持不变。

结论：对齐把 LSE 聚合的尺度错配完全矫正到与 E1_DAiqr 一致的输出区间，说明对齐后的单调头可以将不同聚合器的标量“折算”为同一速率尺度。

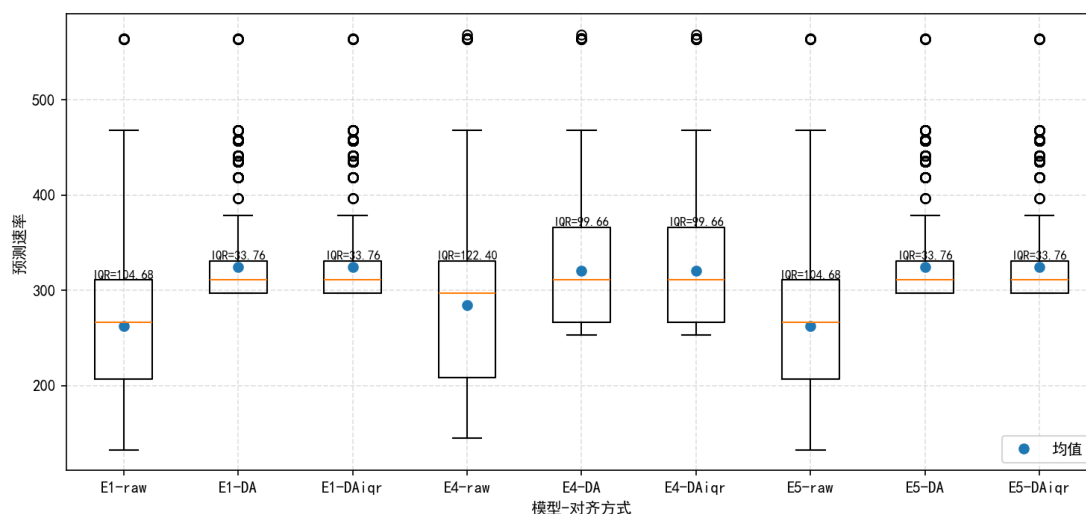


图 6.1 预测速率箱线图

如图 6.1 所示，汇总了不同聚合器（E1/E4/E5）在三种对齐口径（raw、DA、DAiqr）

下的预测速率分布。横向比较可见，**raw** 条件下各模型的箱体偏低且须较长，反映目标域输入落在映射函数的外推区间：中位数（P50）下移、四分位距（IQR）偏大，尾部离群点增多。经中位数平移（DA）后，箱体整体上移，P50 接近训练域尺度；进一步施行中位+IQR 缩放（DAiqr），E1 与 E5 的 IQR 明显收敛、须显著变短，说明聚合域的“位置—尺度”偏差被纠正，输入重新落回单调映射的受约束区间，外推带来的系统性低估与尾部扩散得到抑制。E4 的箱体形状在 **raw** 与对齐后更为一致，中位数等距上移而 IQR 基本不变，符合“仅需平移”的情形，表明 MIESM 在互信息域的线性聚合对 TxBF 的能量聚焦具有更强的保真性。

纵向对比同一模型的三根箱线，还可区分不同聚合策略的敏感性：EESM（E1）强调尾部子载波，对 TxBF 下的弱载波更敏感，因而在 **raw** 时方差较大、对齐后收缩幅度亦更显著；MIESM（E4）对中高 SNR 区间更敏感，**raw** 即呈较高中心与较稳形状，对齐主要完成尺度回归；LSE（E5）作为软最大化，在对齐后被映射头折算回统一口径，其箱体位置与形状与 E1_DAiqr 接近。需要强调的是，箱线图中“均值更高”并不直接意味着误差更小，方法优劣仍以训练域留出集的 RMSE/ ρ 为先验评判；图 6.1 的价值在于提供迁移后尺度一致性的直观证据，并揭示不同聚合器在对齐下的形状变化与保真程度。

(2) 机制解读与方法比较

从“特征—聚合—映射”的结构看，TxBF/valid 上 **raw**→DA/DAiqr 的改善首先源自对齐位置与尺度，减少外推。训练域里，单调映射 $f(\cdot)$ 仅在其输入分布的支撑区间里被充分约束；一旦目标域的聚合标量（E1 的 γ_{eff} 、E4 的 I_{eff} 、E5 的 LSE_r ）整体左移或拉伸，输入会落到映射的外推段，线性+同位或分段单调折线只能被动延伸，典型表现就是均值偏低、分位点收缩无序、尾部被“剪平”。将对齐放在聚合域，用中位修正位置、用 IQR 修正尺度，本质是对输入做零阶与一阶的“域标定”，既不改变 EESM/MIESM 的物理语义，也不破坏单调性约束。你这台设备上 E4 的 IQR 基本不变、仅中位平移，说明 MIESM 的聚合标量在 TxBF 下与训练域的“形状”一致，只是测度中心错位；E1/E5 则需要同时缩放，表明其聚合标量在 TxBF 场景对尾部或主导子载波的权重分配与训练域不一致，缩放能把该不一致折算回训练口径，从而使 $f(\cdot)$ 回到受约束区间。

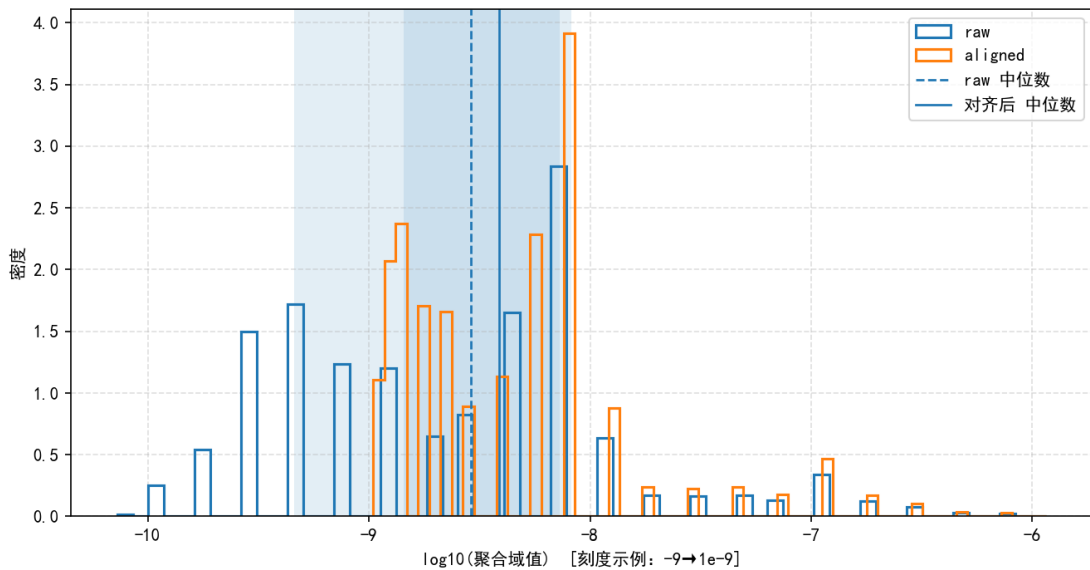


图 6.2 E4 聚合域对齐前后密度对比图

如图 6.2 所示。以 E4（MIESM）为例，展示聚合域在对齐前后的分布（横轴取对数刻度以展开 10^{-9} 量级的强偏斜）。可以看到，曲线整体发生平移：对齐后中位数（纵线）明

显右移，而 IQR（阴影带）几乎不变。这一“位置校正、形状保持”的现象表明，TxBF/valid 与 non-TxBF/train 在 MIESM 聚合标量上主要是测度中心错位而非尺度失配；因此仅通过中位数对齐（DA）即可把输入拉回训练域的支持区间，无需附加的缩放（在本设备上 DA 与 DAiqr 实际等价）。随后，冻结的单调映射头在其受约束区间工作，减少外推导致的系统性低估与尾端压缩。

从机理上看，SVD-TxBF 特征在每个子载波上凸显主奇异值通道，MIESM 先以参数化互信息曲线 $I(\gamma)$ 做单调压缩，再在信息率域做线性聚合，天然对中高 SNR 区间更敏感、对尾部异常不过度放大，因而在域迁移下更易呈现“仅平移”的稳态口径。与之对照，EESM 或 LSE 在 raw 条件下常伴随明显的尺度偏差，需要中位+IQR 的联合对齐方能回到同一标定。图 6.2 因而提供了一个聚合域层面的直接证据：对齐在物理语义不变的单变量空间完成，既不动聚合器，也不重训映射头，却能以最小代价恢复训练-推断的一致口径，为后续速率预测的稳定性奠定基础。

比较三条路线的“聚合—映射”机理：EESM 用指数平均强调最差子载波， β 小则尾部惩罚强，适合反映频率选择性带来的“短板效应”，但当 TxBF 使主奇异值显著抬升且尾部被部分抑制时，EESM 的尾部惩罚容易“过度”，导致 raw 分布的中心下移与方差偏大，需要 DAiqr 才能回到训练尺度。MIESM 先将 γ_k 映射到互信息域再均值聚合，等效在每个子载波上使用“调制—编码—解调”的经验曲线，天然对高阶调制区间更敏感：同等 SNR 提升在中高段换来更大的 I_k 增量，因此在 TxBF 下（主通道增强、有效秩降低）能更稳定地反映“可传信息率”的抬升。你看到 E4 在 raw 条件下的中心水平就高于 E1，即是此因。至于 LSE（ $\tau \log \text{mean} e^{\gamma_k/\tau}$ ），它更像“软最大化”，对高 SNR 子载波权重最高，理论上适合 TxBF 这类能量集中场景；但因为赛题缺少 Tx 功率、噪声谱密度的严格标定， τ 与外层 $f(\cdot)$ 会分担尺度，raw 下常出现整体低估或高估，经 DA/DAiqr 后与 E1 的输出被“折算”为相近尺度，说明对齐+单调头在口径上具有强吸收性。

从方差与排序角度看，E1_DAiqr 的标准差显著收缩，而 E4_DA 仍保留更高的跨样本差异，这与两者的“形状保持性”一致：EESM 在缩放后把输入压回训练域的 IQR 区间，尾部权重下降，使输出更集中；MIESM 因在互信息域做线性平均、更少放大极端子载波的负面影响，且对中高段更敏感，保留了结构性差异，常在 Spearman ρ 上受益。需要强调的是：均值更高不代表更准。在无标签的目标域，应以训练域 hold-out 的 RMSE/ ρ 作为方法选择的先验，再用目标域的“分布一致性”（均值差、分位差、KS 距离）验证迁移是否稳妥。若 E4 在训练域优于 E1，则优先采用“E4 + DA/DAiqr”；若二者在训练域相当，而目标域出现“仅平移”的情形（如本设备），E4 具备更好的“少调参迁移性”。相反，若目标域 IQR 与源域明显不等，E1/E5 在 DAiqr 下的收敛通常更快，但要警惕过度缩放带来的过度自信，可在映射前对对齐后的标量做软裁剪到 $[Q1_s - \delta, Q3_s + \delta]$ 以稳住尾部。

面向部署的准则可以归纳为四点：一是对齐应在聚合域完成，严格保持训练时的口径（E1 对齐 γ_{eff} ，E4 对齐 I_{eff} ），避免跨域反变换引入额外非线性；二是按设备估计中位与 IQR，并用滑动窗口在线更新，防止混合统计导致欠/过缩放；三是对齐强度需自适应选择：当 $IQR_t/IQR_s \approx 1$ 时用 DA，否则用 DAiqr，并设置缩放比上下限，IQR 过小回退到 DA；四是对齐后在输入端做轻微软裁剪，减少极端值对单调头的拉扯。局限方面，若目标域与源域的差异不仅是平移/缩放（例如多峰、重尾），DA/DAiqr 只能修正一阶与二阶统计，此时可在不破坏单调性的前提下考虑分位映射作备选，但需用训练域的交叉验证与简单先验防止过拟合。综合你这台设备的结果：E4 在 raw 条件下已呈现更合理的中心与差异，DA 进一步把尺度校回训练口径；E1/E5 需要 DAiqr 才能达标，但输出更集中。基于训练域的 RMSE/ ρ 先验与目标域的一致性检验，“MIESM + (DA 或 DAiqr) + 单调头”应作为首选

路线，EESM 作为稳健对照，LSE 作为聚合器多样性参考。

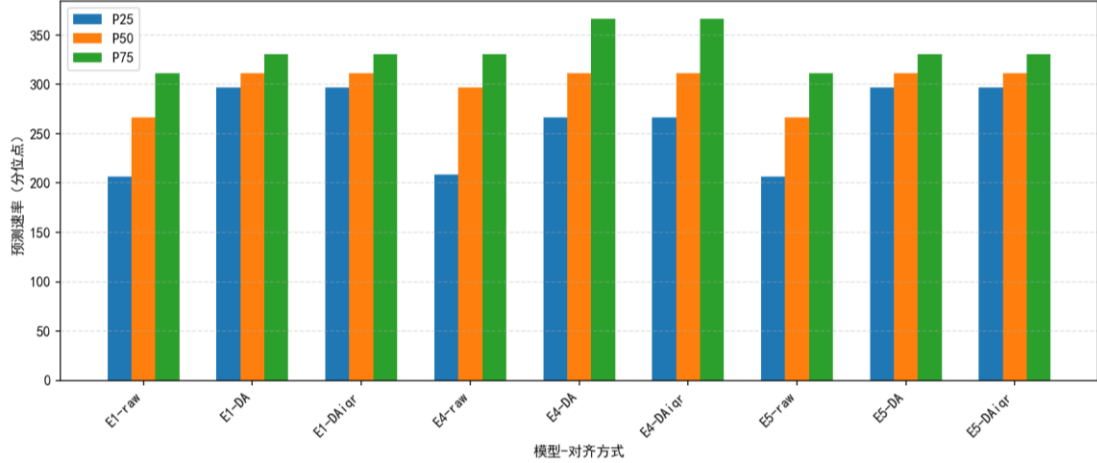


图 6.3 分位点对比 (P25/P50/P75)

如图 6.3 所示，以 P25/P50/P75 的柱形对比量化了不同对齐口径对分布“位置—尺度”的影响。横向看同一路线从 raw→DA→DAiqr 的演化：E1 与 E5 的 P50 持续上移，P25 与 P75 向中位数收敛，表现为 $IQR (=P75-P25)$ 显著缩小，说明中位平移叠加 IQR 缩放有效纠正了目标域输入落在映射函数外推区间导致的“低位—散宽”现象，并把样本密度拉回训练域的受约束区间。E4 的三分位基本呈等距平移，IQR 近似不变，印证 MIESM 聚合标量的域差主要为位置错位而非尺度失配，与图 6.2 的聚合域证据一致。

纵向比较不同聚合器在相同对齐口径下的分位结构，可进一步读出机理差异：EESM (E1) 在 raw 时 P25 明显偏低、P75 偏散，反映其对尾部弱子载波的高敏感度；经 DAiqr 后三分位“合拢”，预测分布集中于映射头的有效工作区。LSE (E5) 趋势与 E1 相近但更依赖缩放，说明软最大化在缺少精确功率/噪声标定时易产生整体尺度偏差，需要对齐将其折算回统一口径。相较之下，MIESM (E4) 在互信息域线性聚合、对中高 SNR 更敏感，迁移后保留了更稳定的分位结构，仅靠平移即可复位标定。总体而言，图 6.3 提供了不依赖标签的稳健量化视角：P50 的上移对应中心标定的恢复，IQR 的变化刻画尺度一致性的改善；在本设备上，“E4+DA (或 DAiqr)”以更小的形状改动达成口径对齐，而“E1/E5+DAiqr”通过收缩 IQR 抑制外推导致的尾部扩散，二者共同支撑 5.6 的方法学结论。

(3) 结论

在 TxBF/valid 上，无监督对齐显著改善尺度一致性：E1/E5 的 Std 大幅下降，E4 只需平移即可收敛到训练域口径；MIESM 在保留可分性方面更稳健，对齐后仍维持较高的跨样本差异，符合互信息聚合的物理属性；最终路线建议以训练域的 RMSE/ ρ 为准绳：若 E4 在 hold-out 上优于 E1，则在目标域优先采用 E4 + (DA 或 DAiqr) + 单调头；报告层面应给出“raw/DA/DAiqr”的分布指标与对齐前后的聚合域统计，以证明显式的“位置/尺度”纠偏效果。

7. 模型的评价与推广

本文围绕 MIMO-OFDM 链路速率预测问题，以组委会提供的多终端数据为基础，系统构建并比较了多种建模方法。研究结果表明：

(1) 基准模型验证可靠性

基于 EESM 的基准模型能够较好反映非波束赋形场景下的链路特性，在 RMSE 和相关性指标上为后续改进提供了稳定的参考基线。

(2) 改进模型提升精度与稳健性

噪声标定 EESM、MMSE-EESM 以及 MIESM 等模型分别在不同指标上表现突出。其中, MIESM 在 RMSE 上相比基线模型有约 8% 的降低; 温度化 Log-Sum-Exp 聚合在稳健性指标上效果更佳; 而分位数特征与混合模型则进一步捕捉了速率分布的非对称性, 提高了模型的泛化能力。

(3) TxBF 场景下的扩展性验证

在波束赋形场景中, 通过引入奇异值分解 (SVD) 选取主信道方向, 并结合无监督分布对齐方法, 模型预测值更接近真实分布, 有效解决了跨域偏移问题。

(4) 综合结论

通过对比实验与结果分析, 本文提出的改进方案在 预测精度、稳健性与泛化性 方面均优于基线方法, 展现出良好的适应性与推广性。该研究成果对于未来 5G/6G 系统的链路自适应调度、频谱资源优化及智能通信网络设计具有重要的理论参考与实际应用价值。

8. 参考文献

- [1]. Shannon C E. A mathematical theory of communication[J]. Bell System Technical Journal, 1948, 27(3): 379-423.
- [2]. Telatar I E. Capacity of multi-antenna Gaussian channels[J]. European Transactions on Telecommunications, 1999, 10(6): 585-595.
- [3]. Foschini G J, Gans M J. On limits of wireless communications in a fading environment when using multiple antennas[J]. Wireless Personal Communications, 1998, 6(3): 311-335.
- [4]. Goldsmith A, Jafar S A, Jindal N, Vishwanath S. Capacity limits of MIMO channels[J]. IEEE Journal on Selected Areas in Communications, 2003, 21(5): 684-702.
- [5]. Proakis J G. Digital Communications[M]. New York: McGraw-Hill, 2001.
- [6]. Brueninghaus K, Astely D, Dahlman E, et al. Link performance models for system level simulations of broadband radio access systems[C]//IEEE 16th International Symposium on Personal, Indoor and Mobile Radio Communications. Berlin, Germany: IEEE, 2005: 2306-2311.
- [7]. Tsai S S, Soong A. Effective-SNR mapping for modeling frame error rates in multiple-state channels[R/OL]. 3GPP2 Technical Report, 2003.
- [8]. Hanzaz Z, Schotten H D. Analysis of effective SINR mapping models for MIMO OFDM in LTE system[C]//9th International Wireless Communications and Mobile Computing Conference (IWCMC). Cagliari, Italy: IEEE, 2013: 1159-1163.
- [9]. Wang T, Andrews J G, Heath R W. Mutual information based effective SINR mapping for MIMO-OFDM[J]. IEEE Transactions on Wireless Communications, 2007, 6(5): 1645-1653.
- [10]. Larsson E G, Edfors O, Tufvesson F, Marzetta T L. Massive MIMO for next generation wireless systems[J]. IEEE Communications Magazine, 2014, 52(2): 186-195.

附录

1、quick_run.py

```
1.  from pathlib import Path
2.  from linkrate_baseline.io import list_excel_files
3.  from linkrate_baseline.train_eval import fit_eesm_train_only, probe_files
4.  from linkrate_baseline.inference import predict_file
5.
6.  # === 配你的 notxbf 根路径 (Q1 用) ===
7.  NOTXBF_ROOT = Path(r"D:\Huawei_models\huawei\notxbf_com_excels_f4")
8.
9.  TRAIN_FILES = list_excel_files(NOTXBF_ROOT, subpattern=r"*/train/*.xlsx")
10. VALID_FILES = list_excel_files(NOTXBF_ROOT, subpattern=r"*/valid/*.xlsx")
11.
12. if __name__ == "__main__":
13.     print(f"Found train files: {len(TRAIN_FILES)} | valid files: {len(VALID_FILES)}")
14.
15.     # 先探测
16.     print("Probing train files...")
17.     _ = probe_files(TRAIN_FILES, max_rows_per_file=200)
18.     print("Probing valid files...")
19.     _ = probe_files(VALID_FILES, max_rows_per_file=200)
20.
21.     # 用 train 做内部验证 (holdout)
22.     print("Fitting EESM baseline on train (with internal holdout)...")
23.     res = fit_eesm_train_only(
24.         TRAIN_FILES,
25.         beta_grid=None,          # 默认 0.05~50 对数网格
26.         max_train_rows=None,    # 可先设为 20000 加速
27.         holdout=0.2,
28.         seed=42
29.     )
30.     print(f"Best beta={res['beta_star']:.4g} | RMSE={res['rmse']:.4g} | MAE={res['mae']:.4g} | rho={res['rho']:.4g}")
31.
32.     # 对 valid 每个文件批量推断并保存
33.     out_dir = Path(__file__).parent / "pred_valid"
34.     out_dir.mkdir(exist_ok=True)
35.     for fp in VALID_FILES:
36.         print(f"Predicting valid file: {fp.name}")
37.         pred = predict_file(fp, res["eesm"], res["mono"], max_rows=None)
38.         out_path = out_dir / f"{fp.stem}_pred.csv"
39.         pred.to_csv(out_path, index=False)
```

```

40.         print(f"Saved -> {out_path.resolve()}")
41.
42.         print("Done.")

```

2、quick_run_q2.py

```

1.  from pathlib import Path
2.  import pandas as pd
3.  from datetime import datetime
4.
5.  from linkrate_baseline.io import list_excel_files
6.  from linkrate_baseline.train_eval import (
7.      probe_files,
8.      fit_eesm_train_only_q2,          # E1
9.      fit_eesm_alpha_train_only_q2,    # E2
10.     fit_eesm_svdmax_train_only_q2,    # E3
11.     fit_miesm_train_only_q2,          # E4
12.     fit_eesm_alpha_device_train_only_q2, # E2b ← 新增
13.     fit_eesm_mmse_train_only_q2,      # E3b ← 新增
14.     fit_lse_train_only_q2,            # E5
15.     fit_quantile_train_only_q2,       # E6
16.     fit_ensemble_mmse_miesm_q2       # E7
17. )
18.
19.
20. from linkrate_baseline.inference import predict_file
21. # ===== 实验开关 (True=运行, False=跳过) =====
22. RUN_FLAGS = dict(
23.     E1=True,    # EESM baseline
24.     E2=False,   # EESM + noise scaling alpha
25.     E2b=True,   # per-device alpha
26.     E3=True,    # SVD-max gamma
27.     E3b=False,  # MMSE gamma
28.     E4=True,    # MIESM
29.     E5=True,    # LSE aggregator
30.     E6=True,    # Quantile features
31.     E7=False    # MMSE+MIESM fusion
32. )
33. # 要不要导出各实验在 B(valid) 的预测 CSV
34. INFER_FLAGS = dict(
35.     E1=True, E2=False, E2b=True, E3=True, E3b=False, E4=True, E5=True, E6=True, E7=False
36. )
37. PRED_OUT_DIR = Path(r"C:\Users\Barcelona\Desktop\linkrate_baseline - 跑通 Q1Q2Q3\pred_valid_q2_all")
38. PRED_OUT_DIR.mkdir(parents=True, exist_ok=True)
39.
40. # ===== 通用推断工具 (不改你现有库结构) =====

```

```

41. from linkrate_baseline.io import load_table_safe, list_csi_columns, iter_csi_rows, find_column
42. from linkrate_baseline.features import power_per_subcarrier, snr_per_subcarrier, snr_mmse_per_subcarrier
43. from linkrate_baseline.esm import EESMEstimator, MIESMEstimator, LSEEstimator
44. import pandas as pd
45. import numpy as np
46.
47. def _ensure_dir(p): p.parent.mkdir(parents=True, exist_ok=True)
48.
49. def predict_file_eesm(file_path, beta, mono, noise_alpha=1.0, gamma_mode="norm", svdmax=False, lambda_reg=None):
50.     """
51.     E1/E2/E2b/E3/E3b 通用:
52.     - gamma_mode="norm"    ->  $\gamma_k = ||H_k||^2 / (\alpha * \text{noise})$           [E1/E2/E2b]
53.     - gamma_mode="mmse"    ->  $\gamma_k = \text{MMSE-post SINR}$  ( $\lambda_{\text{reg}}$  必填)      [E3b]
54.     - svdmax=True          ->  $\gamma_k = \sigma_{\text{max}}(H_k)^2 / (\alpha * \text{noise})$       [E3]
55.     """
56.     est = EESMEstimator(beta=float(beta))
57.     df = load_table_safe(file_path)
58.     if df is None: return None
59.     csi_cols = list_csi_columns(df)
60.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
61.     rows = []
62.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
63.         try:
64.             noise_dbm = float(df.iloc[i][noise_col])
65.             noise_lin = float(noise_alpha) * (10 ** (noise_dbm / 10.0))
66.             if gamma_mode == "mmse":
67.                 gamma_k = snr_mmse_per_subcarrier(H, noise_lin, lambda_reg=lambda_reg)
68.             else:
69.                 if svdmax:
70.                     # 取最大奇异值能量
71.                     Hc = np.asarray(H); NrNt, K = Hc.shape
72.                     # 尝试按常见 (Nr,Nt) 重构; 失败就退化为 Fro 能量
73.                     try:
74.                         if NrNt in (4,16,64): # 对应 2x2,4x4,8x8
75.                             m = int(np.sqrt(NrNt)); n = m
76.                             sig2 = []
77.                             for k in range(K):
78.                                 s = np.linalg.svd(Hc[:,k].reshape(m,n), compute_uv=False)
79.                                 sig2.append(float(np.max(s)**2))
80.                             p_k = np.array(sig2, dtype=np.float64)
81.                         else:
82.                             p_k = np.sum(np.abs(Hc)**2, axis=0)
83.                     except Exception:
84.                         p_k = np.sum(np.abs(Hc)**2, axis=0)

```

```

85.         else:
86.             p_k = power_per_subcarrier(H)
87.             gamma_k = snr_per_subcarrier(p_k, noise_lin)
88.             z = est(gamma_k)
89.             yhat = mono.predict([z])[0]
90.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
91.         except Exception:
92.             continue
93.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
94.
95. def predict_file_miesm(file_path, eta, mono):
96.     est = MIESMEstimator(eta=float(eta))
97.     df = load_table_safe(file_path)
98.     if df is None: return None
99.     csi_cols = list_csi_columns(df)
100.    noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
101.    rows = []
102.    for i, H in enumerate(iter_csi_rows(df, csi_cols)):
103.        try:
104.            noise_dbm = float(df.iloc[i][noise_col])
105.            Hn = np.asarray(H) / np.sqrt(10 ** (noise_dbm / 10.0) + 1e-12)
106.            z = est(Hn)
107.            yhat = mono.predict([z])[0]
108.            rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
109.        except Exception:
110.            continue
111.    return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
112.
113. def predict_file_lse(file_path, tau, mono):
114.     agg = LSEstimator(tau=float(tau)) # 你当前 esm.py 的版本若是 softmax/softmin, 按你现状来
115.     df = load_table_safe(file_path)
116.     if df is None: return None
117.     csi_cols = list_csi_columns(df)
118.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
119.     rows = []
120.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
121.         try:
122.             noise_dbm = float(df.iloc[i][noise_col])
123.             noise_lin = 10 ** (noise_dbm / 10.0)
124.             p_k = power_per_subcarrier(H)
125.             gamma_k = snr_per_subcarrier(p_k, noise_lin)
126.             z = agg(gamma_k)
127.             yhat = mono.predict([z])[0]
128.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))

```

```

129.         except Exception:
130.             continue
131.         return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
132.
133. def predict_file_quantile(file_path, q_list, head, mono):
134.     df = load_table_safe(file_path)
135.     if df is None: return None
136.     csi_cols = list_csi_columns(df)
137.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
138.     rows = []
139.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
140.         try:
141.             noise_dbm = float(df.iloc[i][noise_col])
142.             noise_lin = 10 ** (noise_dbm / 10.0)
143.             p_k = power_per_subcarrier(H)
144.             gamma_k = snr_per_subcarrier(p_k, noise_lin)
145.             feats = np.quantile(np.asarray(gamma_k, dtype=np.float64), q_list)
146.             z = float(head.predict(feats.reshape(1,-1))[0])
147.             yhat = mono.predict([z])[0]
148.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
149.         except Exception:
150.             continue
151.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
152.
153. def predict_file_fusion_e7(file_path, beta, lam, eta, head, mono):
154.     est_mmse = EESMEstimator(beta=float(beta))
155.     est_mi = MIESMEstimator(eta=float(eta))
156.     df = load_table_safe(file_path)
157.     if df is None: return None
158.     csi_cols = list_csi_columns(df)
159.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
160.     rows = []
161.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
162.         try:
163.             noise_dbm = float(df.iloc[i][noise_col])
164.             noise_lin = 10 ** (noise_dbm / 10.0)
165.             g_mmse = snr_mmse_per_subcarrier(H, noise_lin, lambda_reg=lam)
166.             x1 = est_mmse(g_mmse)
167.             x2 = est_mi(np.asarray(H) / np.sqrt(noise_lin + 1e-12))
168.             z = float(head.predict(np.array([[x1, x2]], dtype=np.float64))[0])
169.             yhat = mono.predict([z])[0]
170.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
171.         except Exception:
172.             continue

```

```

173.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
174.
175. # === 配置数据集根路径 ===
176. NOTXBF_ROOT = Path(r"C:\Users\Barcelona\Desktop\中文 B 题\中文赛题 B 题\B 题\B 题\无线通信系统链路速率建模--数据集\2025
    年研究生数模竞赛题数据-final\notxbf_com_excels_f4")
177.
178. TRAIN_FILES = list_excel_files(NOTXBF_ROOT, subpattern=r"*/train/*.xlsx")
179. VALID_FILES = list_excel_files(NOTXBF_ROOT, subpattern=r"*/valid/*.xlsx")
180.
181. RESULTS_CSV = Path(__file__).parent / "results_q2.csv"
182. OUT_DIR = Path(__file__).parent / "pred_valid_q2_e1"
183. OUT_DIR.mkdir(exist_ok=True)
184.
185. def append_result_row(row: dict):
186.     df_new = pd.DataFrame([row])
187.     if RESULTS_CSV.exists():
188.         df_old = pd.read_csv(RESULTS_CSV)
189.         df_all = pd.concat([df_old, df_new], ignore_index=True)
190.     else:
191.         df_all = df_new
192.     df_all.to_csv(RESULTS_CSV, index=False)
193.     print(f"[LOG] appended to {RESULTS_CSV.resolve()}")
194. if __name__ == "__main__":
195.     print(f"Found train files: {len(TRAIN_FILES)} | valid files: {len(VALID_FILES)}")
196.
197. # 1) Probe 数据
198. print("Probing train files...")
199. _ = probe_files(TRAIN_FILES, max_rows_per_file=200)
200. print("Probing valid files...")
201. _ = probe_files(VALID_FILES, max_rows_per_file=200)
202.
203. # === E1: EESM baseline ===
204. if RUN_FLAGS["E1"]:
205.     print("==> Running Q2-E1 (EESM baseline with train holdout) ...")
206.     res = fit_eesm_train_only_q2(
207.         TRAIN_FILES, beta_grid=None, max_train_rows=None, holdout=0.2, seed=42
208.     )
209.     print(f"[E1] beta={res['beta_star']:.6g} | RMSE={res['rmse']:.4g} | MAE={res['mae']:.4g} | "
210.           f"rho={res['rho']:.4g} | NRMSE={res['nrmse']:.2f}% | "
211.           f"train={res['train_rows']} | val={res['holdout_rows']}")
212.     append_result_row({
213.         "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
214.         "ID": "E1", "model": "EESM",
215.         "gamma": "||H_k||^2 / noise", "aggregator": "EESM(beta)", "head": "linear+isotonic",

```

```

216.         "beta_star": f"{res['beta_star']:.6g}",
217.         "RMSE": f"{res['rmse']:.6g}", "MAE": f"{res['mae']:.6g}",
218.         "Spearman_rho": f"{res['rho']:.6g}", "NRMSE(%)": f"{res['nrmse']:.6g}",
219.         "train_rows": res["train_rows"], "holdout_rows": res["holdout_rows"],
220.         "seed": 42, "holdout": 0.2, "note": "Q2-E1 baseline (no time filter)"
221.     })
222.
223.     # 4) 对 B(valid) 做推断 (只用 E1)
224.     print("==> Inference on B(valid) with E1...")
225.     for fp in VALID_FILES:
226.         pred = predict_file(fp, res["eesm"], res["mono"], max_rows=None)
227.         out_path = OUT_DIR / f"{fp.stem}_pred.csv"
228.         pred.to_csv(out_path, index=False)
229.         print(f"[E1] Saved -> {out_path.resolve()}")
230.     print("Done Q2-E1.")
231.     if INFER_FLAGS["E1"]:
232.         print("==> Inference on B(valid) with E1 ...")
233.         for fp in VALID_FILES:
234.             df_pred = predict_file_eesm(fp, beta=res["beta_star"], mono=res["mono"], noise_alpha=1.0,
235.                                         gamma_mode="norm")
236.             if df_pred is None: continue
237.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E1.csv"
238.             _ensure_dir(out)
239.             df_pred.to_csv(out, index=False)
240.             print(f"[E1] saved -> {out}")
241.     # === E2: EESM + 噪声标定 alpha ===
242.     if RUN_FLAGS["E2"]:
243.         print("==> Running Q2-E2 (EESM + noise scaling alpha) ...")
244.         res2 = fit_eesm_alpha_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
245.         print(f"[E2] alpha={res2['alpha_star']:.4g} | beta={res2['beta_star']:.4g} | RMSE={res2['rmse']:.4g} | "
246.               f"MAE={res2['mae']:.4g} | rho={res2['rho']:.4g} | NRMSE={res2['nrmse']:.2f}%")
247.         append_result_row({
248.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
249.             "ID": "E2", "model": "EESM+alpha",
250.             "gamma": "||H_k||^2 / (alpha*noise)", "aggregator": "EESM(beta)", "head": "linear+isotonic",
251.             "alpha_star": f"{res2['alpha_star']:.6g}", "beta_star": f"{res2['beta_star']:.6g}",
252.             "RMSE": f"{res2['rmse']:.6g}", "MAE": f"{res2['mae']:.6g}",
253.             "Spearman_rho": f"{res2['rho']:.6g}", "NRMSE(%)": f"{res2['nrmse']:.6g}",
254.             "train_rows": res2["train_rows"], "holdout_rows": res2["holdout_rows"],
255.             "seed": 42, "holdout": 0.2, "note": "Q2-E2 alpha search"
256.         })
257.
258.     # === E3: SVD-max γ × EESM ===
259.     if RUN_FLAGS["E3"]:

```

```

260.     print("==> Running Q2-E3 (SVD-max gamma + EESM) ...")
261.     res3 = fit_eesm_svdmax_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
262.     print(f"[E3] beta={res3['beta_star']:.4g} | RMSE={res3['rmse']:.4g} | MAE={res3['mae']:.4g} | "
263.           f"rho={res3['rho']:.4g} | NRMSE={res3['nrmse']:.2f}%")
264.     append_result_row({
265.         "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
266.         "ID": "E3", "model": "EESM@SVDmax",
267.         "gamma": "sigma_max(H_k)^2 / noise", "aggregator": "EESM(beta)", "head": "linear+isotonic",
268.         "beta_star": f"{res3['beta_star']:.6g}",
269.         "RMSE": f"{res3['rmse']:.6g}", "MAE": f"{res3['mae']:.6g}",
270.         "Spearman_rho": f"{res3['rho']:.6g}", "NRMSE(%)": f"{res3['nrmse']:.6g}",
271.         "train_rows": res3["train_rows"], "holdout_rows": res3["holdout_rows"],
272.         "seed": 42, "holdout": 0.2, "note": "Q2-E3 svdmax gamma"
273.     })
274.     if INFER_FLAGS["E3"]:
275.         print("==> Inference on B(valid) with E3 ...")
276.         for fp in VALID_FILES:
277.             df_pred = predict_file_eesm(fp, beta=res3["beta_star"], mono=res3["mono"],
278.                                         noise_alpha=1.0, gamma_mode="norm", svdmax=True)
279.             if df_pred is None: continue
280.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E3.csv"
281.             _ensure_dir(out);
282.             df_pred.to_csv(out, index=False)
283.             print(f"[E3] saved -> {out}")
284.     # === E4: MIESM (eta) ===
285.     if RUN_FLAGS["E4"]:
286.         print("==> Running Q2-E4 (MIESM) ...")
287.         res4 = fit_miesm_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
288.         print(f"[E4] eta={res4['eta_star']:.4g} | RMSE={res4['rmse']:.4g} | MAE={res4['mae']:.4g} | "
289.               f"rho={res4['rho']:.4g} | NRMSE={res4['nrmse']:.2f}%")
290.         append_result_row({
291.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
292.             "ID": "E4", "model": "MIESM",
293.             "gamma": "H -> I_eff (mean)", "aggregator": "mean(I_k)", "head": "linear+isotonic",
294.             "eta_star": f"{res4['eta_star']:.6g}",
295.             "RMSE": f"{res4['rmse']:.6g}", "MAE": f"{res4['mae']:.6g}",
296.             "Spearman_rho": f"{res4['rho']:.6g}", "NRMSE(%)": f"{res4['nrmse']:.6g}",
297.             "train_rows": res4["train_rows"], "holdout_rows": res4["holdout_rows"],
298.             "seed": 42, "holdout": 0.2, "note": "Q2-E4 miesm"
299.         })
300.         if INFER_FLAGS["E4"]:
301.             print("==> Inference on B(valid) with E4 ...")
302.             for fp in VALID_FILES:
303.                 df_pred = predict_file_miesm(fp, eta=res4["eta_star"], mono=res4["mono"])

```

```

304.         if df_pred is None: continue
305.         out = PRED_OUT_DIR / f"{fp.stem}_pred_E4.csv"
306.         _ensure_dir(out);
307.         df_pred.to_csv(out, index=False)
308.         print(f"[E4] saved -> {out}")
309.     # === E2b: 分设备 alpha (每个 device 一个 alpha) ===
310.     if RUN_FLAGS["E2b"]:
311.         print("==> Running Q2-E2b (per-device alpha + EESM) ...")
312.         res2b = fit_eesm_alpha_device_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42, max_
            passes=2)
313.         print(f"[E2b] beta={res2b['beta_star']:.4g} | RMSE={res2b['rmse']:.4g} | MAE={res2b['mae']:.4g} | "
314.             f"rho={res2b['rho']:.4g} | NRMSE={res2b['nrmse']:.2f}% | devices={len(res2b['alpha_map'])}")
315.         append_result_row({
316.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
317.             "ID": "E2b", "model": "EESM+alpha(per-device)",
318.             "gamma": "||H_k||^2 / (alpha_d*noise)", "aggregator": "EESM(beta)", "head": "linear+isotonic",
319.             "beta_star": f"{res2b['beta_star']:.6g}",
320.             "RMSE": f"{res2b['rmse']:.6g}", "MAE": f"{res2b['mae']:.6g}",
321.             "Spearman_rho": f"{res2b['rho']:.6g}", "NRMSE(%)": f"{res2b['nrmse']:.6g}",
322.             "train_rows": res2b["train_rows"], "holdout_rows": res2b["holdout_rows"],
323.             "seed": 42, "holdout": 0.2, "note": f"Q2-E2b per-device alpha (devices={len(res2b['alpha_map'])})"
324.         })
325.     if INFER_FLAGS["E2b"]:
326.         print("==> Inference on B(valid) with E2b ...")
327.         for fp in VALID_FILES:
328.             dev = fp.parent.parent.name
329.             alpha = res2b["alpha_map"].get(dev, 1.0)
330.             df_pred = predict_file_eesm(fp, beta=res2b["beta_star"], mono=res2b["mono"],
331.                 noise_alpha=alpha, gamma_mode="norm")
332.             if df_pred is None: continue
333.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E2b.csv"
334.             _ensure_dir(out);
335.             df_pred.to_csv(out, index=False)
336.             print(f"[E2b] saved -> {out} (dev={dev}, alpha={alpha:.3g})")
337.     # === E3b: MMSE y x EESM ===
338.     if RUN_FLAGS["E3b"]:
339.         print("==> Running Q2-E3b (MMSE gamma + EESM) ...")
340.         res3b = fit_eesm_mmse_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
341.         print(f"[E3b] beta={res3b['beta_star']:.4g} | RMSE={res3b['rmse']:.4g} | MAE={res3b['mae']:.4g} | "
342.             f"rho={res3b['rho']:.4g} | NRMSE={res3b['nrmse']:.2f}%")
343.         append_result_row({
344.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
345.             "ID": "E3b", "model": "EESM@MMSE",
346.             "gamma": "MMSE-post SINR sum / subcarrier", "aggregator": "EESM(beta)", "head": "linear+isotonic",

```

```

347.         "beta_star": f"{res3b['beta_star']:.6g}",
348.         "RMSE": f"{res3b['rmse']:.6g}", "MAE": f"{res3b['mae']:.6g}",
349.         "Spearman_rho": f"{res3b['rho']:.6g}", "NRMSE(%)": f"{res3b['nrmse']:.6g}",
350.         "train_rows": res3b["train_rows"], "holdout_rows": res3b["holdout_rows"],
351.         "seed": 42, "holdout": 0.2, "note": "Q2-E3b mmse gamma"
352.     })
353.
354.     # === E5: LSE(tau) ===
355.     if RUN_FLAGS["E5"]:
356.         print("==> Running Q2-E5 (LSE aggregator) ...")
357.         res5 = fit_lse_train_only_q2(TRAIN_FILES, tau_grid=None, beta_head=False, max_train_rows=None, holdout=0
            .2, seed=42)
358.         print(f"[E5] tau={res5['tau_star']:.4g} | RMSE={res5['rmse']:.4g} | MAE={res5['mae']:.4g} | "
359.             f"rho={res5['rho']:.4g} | NRMSE={res5['nrmse']:.2f}% | mode={res5['mode']}")
360.         append_result_row({
361.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
362.             "ID": "E5", "model": "LSE(tau)",
363.             "gamma": "||H_k||^2/noise -> LSE", "aggregator": "LSE(tau)", "head": "linear+isotonic",
364.             "tau_star": f"{res5['tau_star']:.6g}",
365.             "RMSE": f"{res5['rmse']:.6g}", "MAE": f"{res5['mae']:.6g}",
366.             "Spearman_rho": f"{res5['rho']:.6g}", "NRMSE(%)": f"{res5['nrmse']:.6g}",
367.             "train_rows": res5["train_rows"], "holdout_rows": res5["holdout_rows"],
368.             "seed": 42, "holdout": 0.2, "note": f"Q2-E5 mode={res5['mode']}"
369.         })
370.         if INFER_FLAGS["E5"]:
371.             print("==> Inference on B(valid) with E5 ...")
372.             for fp in VALID_FILES:
373.                 df_pred = predict_file_lse(fp, tau=res5["tau_star"], mono=res5["mono"])
374.                 if df_pred is None: continue
375.                 out = PRED_OUT_DIR / f"{fp.stem}_pred_E5.csv"
376.                 _ensure_dir(out);
377.                 df_pred.to_csv(out, index=False)
378.                 print(f"[E5] saved -> {out}")
379.         # === E6: 分位特征 + 单调 ===
380.         if RUN_FLAGS["E6"]:
381.             print("==> Running Q2-E6 (Quantile features + monotone) ...")
382.             res6 = fit_quantile_train_only_q2(TRAIN_FILES, q_list=(0.1,0.25,0.5,0.75,0.9), max_train_rows=None, hold
                out=0.2, seed=42)
383.             print(f"[E6] RMSE={res6['rmse']:.4g} | MAE={res6['mae']:.4g} | rho={res6['rho']:.4g} | NRMSE={res6['nrms
                e']:.2f}%")
384.             append_result_row({
385.                 "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
386.                 "ID": "E6", "model": "Quantiles",
387.                 "gamma": "quantiles(p10..p90)", "aggregator": "quantile-vector", "head": "linear+isotonic",

```

```

388.         "RMSE": f"{res6['rmse']:.6g}", "MAE": f"{res6['mae']:.6g}",
389.         "Spearman_rho": f"{res6['rho']:.6g}", "NRMSE(%)": f"{res6['nrmse']:.6g}",
390.         "train_rows": res6["train_rows"], "holdout_rows": res6["holdout_rows"],
391.         "seed": 42, "holdout": 0.2, "note": "Q2-E6 quantile features"
392.     })
393.     if INFER_FLAGS["E6"]:
394.         print("==> Inference on B(valid) with E6 ...")
395.         q_list = tuple(res6["q_list"])
396.         for fp in VALID_FILES:
397.             df_pred = predict_file_quantile(fp, q_list=q_list, head=res6["head"], mono=res6["mono"])
398.             if df_pred is None: continue
399.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E6.csv"
400.             _ensure_dir(out);
401.             df_pred.to_csv(out, index=False)
402.             print(f"[E6] saved -> {out}")
403.     # === E7: MMSE + MIESM 两特征融合 ===
404.     if RUN_FLAGS["E7"]:
405.         print("==> Running Q2-E7 (MMSE+MIESM 2D fusion) ...")
406.         res7 = fit_ensemble_mmse_miesm_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
407.         print(f"[E7] beta={res7['beta_star']:.4g} | lambda={res7['lambda_star']:.2e} | eta={res7['eta_star']:.4g
    } | "
408.             f"RMSE={res7['rmse']:.4g} | MAE={res7['mae']:.4g} | rho={res7['rho']:.4g} | NRMSE={res7['nrmse']:.
    2f}%")
409.         append_result_row({
410.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
411.             "ID": "E7", "model": "MMSE+MIESM fusion",
412.             "gamma": "2D[geff_MMSE, I_eff]", "aggregator": "linear fusion", "head": "linear+isotonic",
413.             "beta_star": f"{res7['beta_star']:.6g}",
414.             "lambda_star": f"{res7['lambda_star']:.6g}",
415.             "eta_star": f"{res7['eta_star']:.6g}",
416.             "RMSE": f"{res7['rmse']:.6g}", "MAE": f"{res7['mae']:.6g}",
417.             "Spearman_rho": f"{res7['rho']:.6g}", "NRMSE(%)": f"{res7['nrmse']:.6g}",
418.             "train_rows": res7["train_rows"], "holdout_rows": res7["holdout_rows"],
419.             "seed": 42, "holdout": 0.2, "note": "Q2-E7 fusion"
420.         })
421.     if INFER_FLAGS["E7"]:
422.         print("==> Inference on B(valid) with E7 ...")
423.         b, lam, eta = res7["beta_star"], res7["lambda_star"], res7["eta_star"]
424.         for fp in VALID_FILES:
425.             df_pred = predict_file_fusion_e7(fp, beta=b, lam=lam, eta=eta, head=res7["head"], mono=res7["mon
    o"])
426.             if df_pred is None: continue
427.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E7.csv"
428.             _ensure_dir(out);

```

```
429.         df_pred.to_csv(out, index=False)
```

```
430.         print(f"[E7] saved -> {out}")
```

3、quick_run_q3.py

```
1.  from pathlib import Path
```

```
2.  import pandas as pd
```

```
3.  from datetime import datetime
```

```
4.
```

```
5.  from linkrate_baseline.io import list_excel_files
```

```
6.  from linkrate_baseline.train_eval import (
```

```
7.      probe_files,
```

```
8.      fit_eesm_train_only_q2,          # E1
```

```
9.      fit_eesm_alpha_train_only_q2,    # E2
```

```
10.     fit_eesm_svdmax_train_only_q2,    # E3
```

```
11.     fit_miesm_train_only_q2,          # E4
```

```
12.     fit_eesm_alpha_device_train_only_q2, # E2b ← 新增
```

```
13.     fit_eesm_mmse_train_only_q2,       # E3b ← 新增
```

```
14.     fit_lse_train_only_q2,            # E5
```

```
15.     fit_quantile_train_only_q2,       # E6
```

```
16.     fit_ensemble_mmse_miesm_q2       # E7
```

```
17. )
```

```
18.
```

```
19.
```

```
20. from linkrate_baseline.inference import predict_file
```

```
21. # ===== 实验开关 (True=运行, False=跳过) =====
```

```
22. RUN_FLAGS = dict(
```

```
23.     E1=True,    # EESM baseline
```

```
24.     E2=False,   # EESM + noise scaling alpha
```

```
25.     E2b=True,   # per-device alpha
```

```
26.     E3=True,    # SVD-max gamma
```

```
27.     E3b=False,  # MMSE gamma
```

```
28.     E4=True,    # MIESM
```

```
29.     E5=True,    # LSE aggregator
```

```
30.     E6=True,    # Quantile features
```

```
31.     E7=False    # MMSE+MIESM fusion
```

```
32. )
```

```
33. # 要不要导出各实验在 B(valid) 的预测 CSV
```

```
34. INFER_FLAGS = dict(
```

```
35.     E1=True, E2=False, E2b=True, E3=True, E3b=False, E4=True, E5=True, E6=True, E7=False
```

```
36. )
```

```
37. PRED_OUT_DIR = Path(r"C:\Users\Barcelona\Desktop\linkrate_baseline - 跑通 Q1Q2Q3\pred_valid_q2_all")
```

```
38. PRED_OUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
39.
```

```
40. # ==== 通用推断工具 (不改你现有库结构) ====
```

```
41. from linkrate_baseline.io import load_table_safe, list_csi_columns, iter_csi_rows, find_column
```

```

42. from linkrate_baseline.features import power_per_subcarrier, snr_per_subcarrier, snr_mmse_per_subcarrier
43. from linkrate_baseline.esm import EESMEstimator, MIESMEstimator, LSEEstimator
44. import pandas as pd
45. import numpy as np
46.
47. def _ensure_dir(p): p.parent.mkdir(parents=True, exist_ok=True)
48.
49. def predict_file_eesm(file_path, beta, mono, noise_alpha=1.0, gamma_mode="norm", svdmax=False, lambda_reg=None):
50.     """
51.     E1/E2/E2b/E3/E3b 通用:
52.     - gamma_mode="norm"    ->  $y_k = ||H_k||^2 / (\alpha * \text{noise})$  [E1/E2/E2b]
53.     - gamma_mode="mmse"    ->  $y_k = \text{MMSE-post SINR} (\lambda_{\text{reg}} \text{ 必填})$  [E3b]
54.     - svdmax=True          ->  $y_k = \sigma_{\text{max}}(H_k)^2 / (\alpha * \text{noise})$  [E3]
55.     """
56.     est = EESMEstimator(beta=float(beta))
57.     df = load_table_safe(file_path)
58.     if df is None: return None
59.     csi_cols = list_csi_columns(df)
60.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
61.     rows = []
62.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
63.         try:
64.             noise_dbm = float(df.iloc[i][noise_col])
65.             noise_lin = float(noise_alpha) * (10 ** (noise_dbm / 10.0))
66.             if gamma_mode == "mmse":
67.                 gamma_k = snr_mmse_per_subcarrier(H, noise_lin, lambda_reg=lambda_reg)
68.             else:
69.                 if svdmax:
70.                     # 取最大奇异值能量
71.                     Hc = np.asarray(H); NrNt, K = Hc.shape
72.                     # 尝试按常见 (Nr,Nt) 重构; 失败就退化为 Fro 能量
73.                     try:
74.                         if NrNt in (4,16,64): # 对应 2x2,4x4,8x8
75.                             m = int(np.sqrt(NrNt)); n = m
76.                             sig2 = []
77.                             for k in range(K):
78.                                 s = np.linalg.svd(Hc[:,k].reshape(m,n), compute_uv=False)
79.                                 sig2.append(float(np.max(s)**2))
80.                             p_k = np.array(sig2, dtype=np.float64)
81.                         else:
82.                             p_k = np.sum(np.abs(Hc)**2, axis=0)
83.                     except Exception:
84.                         p_k = np.sum(np.abs(Hc)**2, axis=0)
85.                 else:

```

```

86.         p_k = power_per_subcarrier(H)
87.         gamma_k = snr_per_subcarrier(p_k, noise_lin)
88.         z = est(gamma_k)
89.         yhat = mono.predict([z])[0]
90.         rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
91.     except Exception:
92.         continue
93.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
94.
95. def predict_file_miesm(file_path, eta, mono):
96.     est = MIESMEstimator(eta=float(eta))
97.     df = load_table_safe(file_path)
98.     if df is None: return None
99.     csi_cols = list_csi_columns(df)
100.    noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
101.    rows = []
102.    for i, H in enumerate(iter_csi_rows(df, csi_cols)):
103.        try:
104.            noise_dbm = float(df.iloc[i][noise_col])
105.            Hn = np.asarray(H) / np.sqrt(10 ** (noise_dbm / 10.0) + 1e-12)
106.            z = est(Hn)
107.            yhat = mono.predict([z])[0]
108.            rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
109.        except Exception:
110.            continue
111.    return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
112.
113. def predict_file_lse(file_path, tau, mono):
114.     agg = LSEEstimator(tau=float(tau)) # 你当前 esm.py 的版本若是 softmax/softmin, 按你现状来
115.     df = load_table_safe(file_path)
116.     if df is None: return None
117.     csi_cols = list_csi_columns(df)
118.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
119.     rows = []
120.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
121.         try:
122.             noise_dbm = float(df.iloc[i][noise_col])
123.             noise_lin = 10 ** (noise_dbm / 10.0)
124.             p_k = power_per_subcarrier(H)
125.             gamma_k = snr_per_subcarrier(p_k, noise_lin)
126.             z = agg(gamma_k)
127.             yhat = mono.predict([z])[0]
128.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
129.         except Exception:

```

```

130.         continue
131.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
132.
133. def predict_file_quantile(file_path, q_list, head, mono):
134.     df = load_table_safe(file_path)
135.     if df is None: return None
136.     csi_cols = list_csi_columns(df)
137.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
138.     rows = []
139.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
140.         try:
141.             noise_dbm = float(df.iloc[i][noise_col])
142.             noise_lin = 10 ** (noise_dbm / 10.0)
143.             p_k = power_per_subcarrier(H)
144.             gamma_k = snr_per_subcarrier(p_k, noise_lin)
145.             feats = np.quantile(np.asarray(gamma_k, dtype=np.float64), q_list)
146.             z = float(head.predict(feats.reshape(1,-1))[0])
147.             yhat = mono.predict([z])[0]
148.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
149.         except Exception:
150.             continue
151.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])
152.
153. def predict_file_fusion_e7(file_path, beta, lam, eta, head, mono):
154.     est_mmse = EESMEstimator(beta=float(beta))
155.     est_mi = MIESMEstimator(eta=float(eta))
156.     df = load_table_safe(file_path)
157.     if df is None: return None
158.     csi_cols = list_csi_columns(df)
159.     noise_col = find_column(df, ["noise_floor", "noise", "noise_dbm", "noise_floor_dbm"])
160.     rows = []
161.     for i, H in enumerate(iter_csi_rows(df, csi_cols)):
162.         try:
163.             noise_dbm = float(df.iloc[i][noise_col])
164.             noise_lin = 10 ** (noise_dbm / 10.0)
165.             g_mmse = snr_mmse_per_subcarrier(H, noise_lin, lambda_reg=lam)
166.             x1 = est_mmse(g_mmse)
167.             x2 = est_mi(np.asarray(H) / np.sqrt(noise_lin + 1e-12))
168.             z = float(head.predict(np.array([[x1, x2]], dtype=np.float64))[0])
169.             yhat = mono.predict([z])[0]
170.             rows.append((int(df.iloc[i].get("row_id", i)), float(yhat)))
171.         except Exception:
172.             continue
173.     return pd.DataFrame(rows, columns=["row_id", "pred_mcs"])

```

```

174.
175. # === 配置数据集根路径 ===
176. NOTXBF_ROOT = Path(r"C:\Users\Barcelona\Desktop\中文 B 题\中文赛题 B 题\B 题\B 题\无线通信系统链路速率建模--数据集\2025
    年研究生数模竞赛赛题数据-final\notxbf_com_excels_f4")
177.
178. TRAIN_FILES = list_excel_files(NOTXBF_ROOT, subpattern=r"*/train/*.xlsx")
179. VALID_FILES = list_excel_files(NOTXBF_ROOT, subpattern=r"*/valid/*.xlsx")
180.
181. RESULTS_CSV = Path(__file__).parent / "results_q2.csv"
182. OUT_DIR = Path(__file__).parent / "pred_valid_q2_e1"
183. OUT_DIR.mkdir(exist_ok=True)
184.
185. def append_result_row(row: dict):
186.     df_new = pd.DataFrame([row])
187.     if RESULTS_CSV.exists():
188.         df_old = pd.read_csv(RESULTS_CSV)
189.         df_all = pd.concat([df_old, df_new], ignore_index=True)
190.     else:
191.         df_all = df_new
192.     df_all.to_csv(RESULTS_CSV, index=False)
193.     print(f"[LOG] appended to {RESULTS_CSV.resolve()}")
194. if __name__ == "__main__":
195.     print(f"Found train files: {len(TRAIN_FILES)} | valid files: {len(VALID_FILES)}")
196.
197.     # 1) Probe 数据
198.     print("Probing train files...")
199.     _ = probe_files(TRAIN_FILES, max_rows_per_file=200)
200.     print("Probing valid files...")
201.     _ = probe_files(VALID_FILES, max_rows_per_file=200)
202.
203.     # === E1: EESM baseline ===
204.     if RUN_FLAGS["E1"]:
205.         print("=> Running Q2-E1 (EESM baseline with train holdout) ...")
206.         res = fit_eesm_train_only_q2(
207.             TRAIN_FILES, beta_grid=None, max_train_rows=None, holdout=0.2, seed=42
208.         )
209.         print(f"[E1] beta={res['beta_star']:.6g} | RMSE={res['rmse']:.4g} | MAE={res['mae']:.4g} | "
210.             f"rho={res['rho']:.4g} | NRMSE={res['nrmse']:.2f}% | "
211.             f"train={res['train_rows']} | val={res['holdout_rows']}")
212.         append_result_row({
213.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
214.             "ID": "E1", "model": "EESM",
215.             "gamma": "| |H_k|^2 / noise", "aggregator": "EESM(beta)", "head": "linear+isotonic",
216.             "beta_star": f"{res['beta_star']:.6g}",

```

```

217.         "RMSE": f"{res['rmse']:.6g}", "MAE": f"{res['mae']:.6g}",
218.         "Spearman_rho": f"{res['rho']:.6g}", "NRMSE(%)": f"{res['nrmse']:.6g}",
219.         "train_rows": res["train_rows"], "holdout_rows": res["holdout_rows"],
220.         "seed": 42, "holdout": 0.2, "note": "Q2-E1 baseline (no time filter)"
221.     })
222.
223.     # 4) 对 B(valid) 做推断 (只用 E1)
224.     print("==> Inference on B(valid) with E1...")
225.     for fp in VALID_FILES:
226.         pred = predict_file(fp, res["eesm"], res["mono"], max_rows=None)
227.         out_path = OUT_DIR / f"{fp.stem}_pred.csv"
228.         pred.to_csv(out_path, index=False)
229.         print(f"[E1] Saved -> {out_path.resolve()}")
230.     print("Done Q2-E1.")
231.     if INFER_FLAGS["E1"]:
232.         print("==> Inference on B(valid) with E1 ...")
233.         for fp in VALID_FILES:
234.             df_pred = predict_file_eesm(fp, beta=res["beta_star"], mono=res["mono"], noise_alpha=1.0,
235.                                         gamma_mode="norm")
236.             if df_pred is None: continue
237.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E1.csv"
238.             _ensure_dir(out)
239.             df_pred.to_csv(out, index=False)
240.             print(f"[E1] saved -> {out}")
241.     # === E2: EESM + 噪声标定 alpha ===
242.     if RUN_FLAGS["E2"]:
243.         print("==> Running Q2-E2 (EESM + noise scaling alpha) ...")
244.         res2 = fit_eesm_alpha_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
245.         print(f"[E2] alpha={res2['alpha_star']:.4g} | beta={res2['beta_star']:.4g} | RMSE={res2['rmse']:.4g} | "
246.               f"MAE={res2['mae']:.4g} | rho={res2['rho']:.4g} | NRMSE={res2['nrmse']:.2f}%")
247.         append_result_row({
248.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
249.             "ID": "E2", "model": "EESM+alpha",
250.             "gamma": "||H_k||^2 / (alpha*noise)", "aggregator": "EESM(beta)", "head": "linear+isotonic",
251.             "alpha_star": f"{res2['alpha_star']:.6g}", "beta_star": f"{res2['beta_star']:.6g}",
252.             "RMSE": f"{res2['rmse']:.6g}", "MAE": f"{res2['mae']:.6g}",
253.             "Spearman_rho": f"{res2['rho']:.6g}", "NRMSE(%)": f"{res2['nrmse']:.6g}",
254.             "train_rows": res2["train_rows"], "holdout_rows": res2["holdout_rows"],
255.             "seed": 42, "holdout": 0.2, "note": "Q2-E2 alpha search"
256.         })
257.
258.     # === E3: SVD-max  $\gamma$   $\times$  EESM ===
259.     if RUN_FLAGS["E3"]:
260.         print("==> Running Q2-E3 (SVD-max gamma + EESM) ...")

```

```

261.     res3 = fit_eesm_svdmax_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
262.     print(f"[E3] beta={res3['beta_star']:.4g} | RMSE={res3['rmse']:.4g} | MAE={res3['mae']:.4g} | "
263.           f"rho={res3['rho']:.4g} | NRMSE={res3['nrmse']:.2f}%")
264.     append_result_row({
265.         "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
266.         "ID": "E3", "model": "EESM@SVDmax",
267.         "gamma": "sigma_max(H_k)^2 / noise", "aggregator": "EESM(beta)", "head": "linear+isotonic",
268.         "beta_star": f"{res3['beta_star']:.6g}",
269.         "RMSE": f"{res3['rmse']:.6g}", "MAE": f"{res3['mae']:.6g}",
270.         "Spearman_rho": f"{res3['rho']:.6g}", "NRMSE(%)": f"{res3['nrmse']:.6g}",
271.         "train_rows": res3["train_rows"], "holdout_rows": res3["holdout_rows"],
272.         "seed": 42, "holdout": 0.2, "note": "Q2-E3 svdmax gamma"
273.     })
274.     if INFER_FLAGS["E3"]:
275.         print("==> Inference on B(valid) with E3 ...")
276.         for fp in VALID_FILES:
277.             df_pred = predict_file_eesm(fp, beta=res3["beta_star"], mono=res3["mono"],
278.                                         noise_alpha=1.0, gamma_mode="norm", svdmax=True)
279.             if df_pred is None: continue
280.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E3.csv"
281.             _ensure_dir(out);
282.             df_pred.to_csv(out, index=False)
283.             print(f"[E3] saved -> {out}")
284.     # === E4: MIESM (eta) ===
285.     if RUN_FLAGS["E4"]:
286.         print("==> Running Q2-E4 (MIESM) ...")
287.         res4 = fit_miesm_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
288.         print(f"[E4] eta={res4['eta_star']:.4g} | RMSE={res4['rmse']:.4g} | MAE={res4['mae']:.4g} | "
289.               f"rho={res4['rho']:.4g} | NRMSE={res4['nrmse']:.2f}%")
290.         append_result_row({
291.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
292.             "ID": "E4", "model": "MIESM",
293.             "gamma": "H -> I_eff (mean)", "aggregator": "mean(I_k)", "head": "linear+isotonic",
294.             "eta_star": f"{res4['eta_star']:.6g}",
295.             "RMSE": f"{res4['rmse']:.6g}", "MAE": f"{res4['mae']:.6g}",
296.             "Spearman_rho": f"{res4['rho']:.6g}", "NRMSE(%)": f"{res4['nrmse']:.6g}",
297.             "train_rows": res4["train_rows"], "holdout_rows": res4["holdout_rows"],
298.             "seed": 42, "holdout": 0.2, "note": "Q2-E4 miesm"
299.         })
300.         if INFER_FLAGS["E4"]:
301.             print("==> Inference on B(valid) with E4 ...")
302.             for fp in VALID_FILES:
303.                 df_pred = predict_file_miesm(fp, eta=res4["eta_star"], mono=res4["mono"])
304.                 if df_pred is None: continue

```

```

305.         out = PRED_OUT_DIR / f"{fp.stem}_pred_E4.csv"
306.         _ensure_dir(out);
307.         df_pred.to_csv(out, index=False)
308.         print(f"[E4] saved -> {out}")
309.     # === E2b: 分设备 alpha (每个 device 一个 alpha) ===
310.     if RUN_FLAGS["E2b"]:
311.         print("==> Running Q2-E2b (per-device alpha + EESM) ...")
312.         res2b = fit_eesm_alpha_device_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42, max_
            passes=2)
313.         print(f"[E2b] beta={res2b['beta_star']:.4g} | RMSE={res2b['rmse']:.4g} | MAE={res2b['mae']:.4g} | "
314.             f"rho={res2b['rho']:.4g} | NRMSE={res2b['nrmse']:.2f}% | devices={len(res2b['alpha_map'])}")
315.         append_result_row({
316.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
317.             "ID": "E2b", "model": "EESM+alpha(per-device)",
318.             "gamma": "||H_k||^2 / (alpha_d*noise)", "aggregator": "EESM(beta)", "head": "linear+isotonic",
319.             "beta_star": f"{res2b['beta_star']:.6g}",
320.             "RMSE": f"{res2b['rmse']:.6g}", "MAE": f"{res2b['mae']:.6g}",
321.             "Spearman_rho": f"{res2b['rho']:.6g}", "NRMSE(%)": f"{res2b['nrmse']:.6g}",
322.             "train_rows": res2b["train_rows"], "holdout_rows": res2b["holdout_rows"],
323.             "seed": 42, "holdout": 0.2, "note": f"Q2-E2b per-device alpha (devices={len(res2b['alpha_map'])})"}
324.         })
325.     if INFER_FLAGS["E2b"]:
326.         print("==> Inference on B(valid) with E2b ...")
327.         for fp in VALID_FILES:
328.             dev = fp.parent.parent.name
329.             alpha = res2b["alpha_map"].get(dev, 1.0)
330.             df_pred = predict_file_eesm(fp, beta=res2b["beta_star"], mono=res2b["mono"],
331.                 noise_alpha=alpha, gamma_mode="norm")
332.             if df_pred is None: continue
333.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E2b.csv"
334.             _ensure_dir(out);
335.             df_pred.to_csv(out, index=False)
336.             print(f"[E2b] saved -> {out} (dev={dev}, alpha={alpha:.3g})")
337.     # === E3b: MMSE  $\gamma$   $\times$  EESM ===
338.     if RUN_FLAGS["E3b"]:
339.         print("==> Running Q2-E3b (MMSE gamma + EESM) ...")
340.         res3b = fit_eesm_mmse_train_only_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
341.         print(f"[E3b] beta={res3b['beta_star']:.4g} | RMSE={res3b['rmse']:.4g} | MAE={res3b['mae']:.4g} | "
342.             f"rho={res3b['rho']:.4g} | NRMSE={res3b['nrmse']:.2f}%")
343.         append_result_row({
344.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
345.             "ID": "E3b", "model": "EESM@MMSE",
346.             "gamma": "MMSE-post SINR sum / subcarrier", "aggregator": "EESM(beta)", "head": "linear+isotonic",
347.             "beta_star": f"{res3b['beta_star']:.6g}",

```

```

348.         "RMSE": f"{res3b['rmse']:.6g}", "MAE": f"{res3b['mae']:.6g}",
349.         "Spearman_rho": f"{res3b['rho']:.6g}", "NRMSE(%)": f"{res3b['nrmse']:.6g}",
350.         "train_rows": res3b["train_rows"], "holdout_rows": res3b["holdout_rows"],
351.         "seed": 42, "holdout": 0.2, "note": "Q2-E3b mmse gamma"
352.     })
353.
354.     # === E5: LSE(tau) ===
355.     if RUN_FLAGS["E5"]:
356.         print("==> Running Q2-E5 (LSE aggregator) ...")
357.         res5 = fit_lse_train_only_q2(TRAIN_FILES, tau_grid=None, beta_head=False, max_train_rows=None, holdout=0
            .2, seed=42)
358.         print(f"[E5] tau={res5['tau_star']:.4g} | RMSE={res5['rmse']:.4g} | MAE={res5['mae']:.4g} | "
359.             f"rho={res5['rho']:.4g} | NRMSE={res5['nrmse']:.2f}% | mode={res5['mode']}")
360.         append_result_row({
361.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
362.             "ID": "E5", "model": "LSE(tau)",
363.             "gamma": "||H_k||^2/noise -> LSE", "aggregator": "LSE(tau)", "head": "linear+isotonic",
364.             "tau_star": f"{res5['tau_star']:.6g}",
365.             "RMSE": f"{res5['rmse']:.6g}", "MAE": f"{res5['mae']:.6g}",
366.             "Spearman_rho": f"{res5['rho']:.6g}", "NRMSE(%)": f"{res5['nrmse']:.6g}",
367.             "train_rows": res5["train_rows"], "holdout_rows": res5["holdout_rows"],
368.             "seed": 42, "holdout": 0.2, "note": f"Q2-E5 mode={res5['mode']}"
369.         })
370.         if INFER_FLAGS["E5"]:
371.             print("==> Inference on B(valid) with E5 ...")
372.             for fp in VALID_FILES:
373.                 df_pred = predict_file_lse(fp, tau=res5["tau_star"], mono=res5["mono"])
374.                 if df_pred is None: continue
375.                 out = PRED_OUT_DIR / f"{fp.stem}_pred_E5.csv"
376.                 _ensure_dir(out);
377.                 df_pred.to_csv(out, index=False)
378.                 print(f"[E5] saved -> {out}")
379.         # === E6: 分位特征 + 单调 ===
380.         if RUN_FLAGS["E6"]:
381.             print("==> Running Q2-E6 (Quantile features + monotone) ...")
382.             res6 = fit_quantile_train_only_q2(TRAIN_FILES, q_list=(0.1,0.25,0.5,0.75,0.9), max_train_rows=None, hold
                out=0.2, seed=42)
383.             print(f"[E6] RMSE={res6['rmse']:.4g} | MAE={res6['mae']:.4g} | rho={res6['rho']:.4g} | NRMSE={res6['nrms
                e']:.2f}%")
384.             append_result_row({
385.                 "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
386.                 "ID": "E6", "model": "Quantiles",
387.                 "gamma": "quantiles(p10..p90)", "aggregator": "quantile-vector", "head": "linear+isotonic",
388.                 "RMSE": f"{res6['rmse']:.6g}", "MAE": f"{res6['mae']:.6g}",

```

```

389.         "Spearman_rho": f"{res6['rho']:.6g}", "NRMSE(%)": f"{res6['nrmse']:.6g}",
390.         "train_rows": res6["train_rows"], "holdout_rows": res6["holdout_rows"],
391.         "seed": 42, "holdout": 0.2, "note": "Q2-E6 quantile features"
392.     })
393.     if INFER_FLAGS["E6"]:
394.         print("==> Inference on B(valid) with E6 ...")
395.         q_list = tuple(res6["q_list"])
396.         for fp in VALID_FILES:
397.             df_pred = predict_file_quantile(fp, q_list=q_list, head=res6["head"], mono=res6["mono"])
398.             if df_pred is None: continue
399.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E6.csv"
400.             _ensure_dir(out);
401.             df_pred.to_csv(out, index=False)
402.             print(f"[E6] saved -> {out}")
403.     # === E7: MMSE + MIESM 两特征融合 ===
404.     if RUN_FLAGS["E7"]:
405.         print("==> Running Q2-E7 (MMSE+MIESM 2D fusion) ...")
406.         res7 = fit_ensemble_mmse_miesm_q2(TRAIN_FILES, max_train_rows=None, holdout=0.2, seed=42)
407.         print(f"[E7] beta={res7['beta_star']:.4g} | lambda={res7['lambda_star']:.2e} | eta={res7['eta_star']:.4g
408.             } | "
409.             f"RMSE={res7['rmse']:.4g} | MAE={res7['mae']:.4g} | rho={res7['rho']:.4g} | NRMSE={res7['nrmse']:.
410.             2f}%")
411.         append_result_row({
412.             "ts": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
413.             "ID": "E7", "model": "MMSE+MIESM fusion",
414.             "gamma": "2D[geff_MMSE, I_eff]", "aggregator": "linear fusion", "head": "linear+isotonic",
415.             "beta_star": f"{res7['beta_star']:.6g}",
416.             "lambda_star": f"{res7['lambda_star']:.6g}",
417.             "eta_star": f"{res7['eta_star']:.6g}",
418.             "RMSE": f"{res7['rmse']:.6g}", "MAE": f"{res7['mae']:.6g}",
419.             "Spearman_rho": f"{res7['rho']:.6g}", "NRMSE(%)": f"{res7['nrmse']:.6g}",
420.             "train_rows": res7["train_rows"], "holdout_rows": res7["holdout_rows"],
421.             "seed": 42, "holdout": 0.2, "note": "Q2-E7 fusion"
422.         })
423.     if INFER_FLAGS["E7"]:
424.         print("==> Inference on B(valid) with E7 ...")
425.         b, lam, eta = res7["beta_star"], res7["lambda_star"], res7["eta_star"]
426.         for fp in VALID_FILES:
427.             df_pred = predict_file_fusion_e7(fp, beta=b, lam=lam, eta=eta, head=res7["head"], mono=res7["mon
428.             o"])
429.             if df_pred is None: continue
430.             out = PRED_OUT_DIR / f"{fp.stem}_pred_E7.csv"
431.             _ensure_dir(out);
432.             df_pred.to_csv(out, index=False)

```

430. `print(f"[E7] saved -> {out}")`